



**CHALMERS**  
UNIVERSITY OF TECHNOLOGY



UNIVERSITY OF GOTHENBURG

---

# Exploring LLM-Powered Software Testing

A Study on Multi-Agent Systems for REST APIs

Master's thesis in Computer Science and Engineering

EFSTATHIOS SIDIROPOULOS



MASTER'S THESIS 2024

# Exploring LLM-Powered Software Testing

A Study on Multi-Agent Systems for REST APIs

EFSTATHIOS SIDIROPOULOS



UNIVERSITY OF  
GOTHENBURG

---



**CHALMERS**  
UNIVERSITY OF TECHNOLOGY

Department of Computer Science and Engineering  
CHALMERS UNIVERSITY OF TECHNOLOGY  
UNIVERSITY OF GOTHENBURG  
Gothenburg, Sweden 2024

Exploring LLM-Powered Software Testing  
A Study on Multi-Agent Systems for REST APIs  
EFSTATHIOS SIDIROPOULOS

© EFSTATHIOS SIDIROPOULOS, 2024.

Supervisor: Yinan Yu, Department of Computer Science and Engineering  
Supervisor: Robert Feldt, Department of Computer Science and Engineering  
Examiner: Matti Karppa, Department of Computer Science and Engineering

Master's Thesis 2024  
Department of Computer Science and Engineering  
Chalmers University of Technology and University of Gothenburg  
SE-412 96 Gothenburg  
Telephone +46 31 772 1000

Typeset in L<sup>A</sup>T<sub>E</sub>X  
Gothenburg, Sweden 2024

Exploring LLM-Powered Software Testing  
A Study on Multi-Agent Systems for REST APIs  
EFSTATHIOS SIDIROPOULOS  
Department of Computer Science and Engineering  
Chalmers University of Technology and University of Gothenburg

## Abstract

In this thesis, we explore the integration of Large Language Models (LLMs) with Multi-Agent Systems (MAS) to enhance the process of REST API testing. By leveraging the contextual understanding of LLMs, this research aims to develop a framework, which starts by enriching the original API specification, generates test cases and analyzes their results, offering an LLM-powered automated solution for API testing. We begin by examining the current industry practices in API testing, identifying the extent of automation and the specific difficulties faced by developers. Subsequently, we implement and evaluate the enhanced specification across three different API specifications. Results indicate that while LLMs show promise in automating parts of the testing process, there are limitations, particularly in handling the extended length of API specifications. Despite these challenges, the integration of LLM agents in the testing process shows significant potential for reducing manual effort and strengthening software quality assurance.

Keywords: Large Language Models, Multi-Agent Systems, REST API Testing, Automated Testing, API Specifications, Software Quality Assurance



# Acknowledgements

I would like to express my deepest gratitude to my supervisors, Yinan Yu and Robert Feldt, for their invaluable guidance, support and encouragement throughout this thesis project. I am especially grateful to Yinan Yu for the countless hours she has spent mentoring me and providing insightful feedback. Her dedication and patience have been instrumental in shaping this project, and her unwavering support has been truly invaluable.

I am also deeply grateful to the industry professionals who participated in the interviews. Their willingness to share their experience and insights provided an important foundation for this study and strengthened the quality of this research.

Finally, I want to extend my heartfelt thanks to my partner and friends. Their support, patience and understanding have been a source of strength throughout this challenging process. Without their encouragement and belief in me, completing this thesis would not have been possible.

Thank you all.

Efstathios Sidiropoulos, Gothenburg, 2024-11-24



# Contents

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| <b>List of Figures</b>                                                     | <b>x</b>  |
| <b>List of Tables</b>                                                      | <b>xi</b> |
| <b>1 Introduction</b>                                                      | <b>1</b>  |
| 1.1 Problem Formulation . . . . .                                          | 2         |
| 1.2 Goals . . . . .                                                        | 2         |
| <b>2 Background</b>                                                        | <b>5</b>  |
| 2.1 Theoretical Background . . . . .                                       | 5         |
| 2.1.1 RESTful APIs . . . . .                                               | 5         |
| 2.1.2 Testing Methodologies in Software Development . . . . .              | 7         |
| 2.1.3 Challenges in REST API Testing . . . . .                             | 8         |
| 2.1.4 Large Language Models . . . . .                                      | 9         |
| 2.1.5 Multi-Agent Systems . . . . .                                        | 12        |
| 2.2 Related Work . . . . .                                                 | 13        |
| 2.2.1 Empirical Studies on REST API Design and Testing Practices . . . . . | 13        |
| 2.2.2 Rest API testing with LLMs . . . . .                                 | 14        |
| <b>3 Methods</b>                                                           | <b>17</b> |
| 3.1 Approach . . . . .                                                     | 17        |
| 3.1.1 Industry Insights . . . . .                                          | 17        |
| 3.1.2 Leveraging Multi-Agent Systems for REST API Testing . . . . .        | 20        |
| 3.1.3 Implementation Details & LangGraph . . . . .                         | 20        |
| 3.1.4 System Design . . . . .                                              | 22        |
| 3.1.5 Evaluation Setup . . . . .                                           | 25        |
| 3.2 Evaluation of the Results . . . . .                                    | 26        |
| 3.2.1 Evaluation of ‘Spec+’ Documents . . . . .                            | 26        |
| 3.2.2 Evaluation of Test Cases . . . . .                                   | 26        |
| <b>4 Results</b>                                                           | <b>29</b> |
| 4.1 RQ1: Industry Insights . . . . .                                       | 29        |
| 4.2 RQ2: The ‘Spec+’ . . . . .                                             | 31        |
| 4.2.1 The Generation of ‘Spec+’ . . . . .                                  | 31        |
| 4.2.2 The Impact of ‘Spec+’ . . . . .                                      | 34        |
| 4.3 RQ3: Test Cases & Results . . . . .                                    | 36        |

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>5</b> | <b>Conclusion</b>                                                 | <b>41</b> |
| 5.1      | Discussion . . . . .                                              | 41        |
| 5.2      | Future Work . . . . .                                             | 42        |
| 5.3      | Final Remarks . . . . .                                           | 43        |
|          | <b>Bibliography</b>                                               | <b>45</b> |
| <b>A</b> | <b>Interview Questions</b>                                        | <b>I</b>  |
| A.1      | Development Process . . . . .                                     | I         |
| A.2      | Tools and Standards . . . . .                                     | I         |
| A.3      | Testing Process . . . . .                                         | I         |
| A.3.1    | Testing Practices . . . . .                                       | I         |
| A.3.2    | Test Case Development . . . . .                                   | I         |
| A.3.3    | Manual vs. Automated Testing . . . . .                            | I         |
| A.4      | Automation Levels . . . . .                                       | I         |
| A.5      | Key Challenges and Observations . . . . .                         | II        |
| A.5.1    | Challenges . . . . .                                              | II        |
| A.5.2    | Areas for Improvement . . . . .                                   | II        |
| A.6      | Specific Questions That Came Up For Different Companies . . . . . | II        |
| A.6.1    | Company 1 . . . . .                                               | II        |
| A.6.2    | Company 2 . . . . .                                               | II        |
| A.6.3    | Company 3 . . . . .                                               | II        |
| A.6.4    | Company 4 . . . . .                                               | II        |
| A.6.5    | Company 5 . . . . .                                               | III       |
| <b>B</b> | <b>Interview Results</b>                                          | <b>IV</b> |
| <b>C</b> | <b>Agent Prompts</b>                                              | <b>VI</b> |
| C.1      | Team Supervisor . . . . .                                         | VI        |
| C.2      | API Expert . . . . .                                              | VI        |
| C.3      | API Improver . . . . .                                            | VII       |
| C.4      | Reflection Actor . . . . .                                        | VII       |
| C.5      | Reflection Evaluator . . . . .                                    | VII       |
| C.6      | Test Execution Chain . . . . .                                    | VIII      |
| <b>D</b> | <b>Agent Responses</b>                                            | <b>IX</b> |
| D.1      | API Expert . . . . .                                              | IX        |
| D.2      | Evaluator . . . . .                                               | XI        |
| D.3      | Actor . . . . .                                                   | XII       |
| D.3.1    | Negative Testing sample . . . . .                                 | XII       |
| D.3.2    | Boundary Testing sample . . . . .                                 | XIII      |
| D.3.3    | Error Handling sample . . . . .                                   | XIII      |
| D.3.4    | Concurrency Testing sample . . . . .                              | XIV       |
| D.3.5    | Performance Testing sample . . . . .                              | XIV       |
| D.3.6    | Edge Case Testing sample . . . . .                                | XV        |
| D.3.7    | Pagination Testing sample . . . . .                               | XV        |
| D.3.8    | Localization Testing sample . . . . .                             | XVI       |

# List of Figures

|     |                                                                                    |    |
|-----|------------------------------------------------------------------------------------|----|
| 1.1 | Example of an API test case. . . . .                                               | 2  |
| 2.1 | Example of OpenAPI Specification (OpenAPI, 2021). . . . .                          | 7  |
| 2.2 | The Transformer architecture (Vaswani et al., 2017) . . . . .                      | 10 |
| 2.3 | Action Space in the LLM knowledge base . . . . .                                   | 15 |
| 3.1 | Dimensions of interviewed companies . . . . .                                      | 18 |
| 3.2 | Illustration of conditional edge usage (LangChain, 2024b). . . . .                 | 22 |
| 3.3 | Specification Enhancement Team . . . . .                                           | 23 |
| 3.4 | API Testing Team . . . . .                                                         | 25 |
| 3.5 | API Testing Workflow . . . . .                                                     | 25 |
| 3.6 | Test coverage levels (Martin-Lopez et al., 2019) . . . . .                         | 26 |
| 4.1 | The ‘markets’ endpoint from the original specification of the Spotify API. . . . . | 31 |
| 4.2 | The ‘markets’ endpoint from the generated ‘Spec+’ of the Spotify API. . . . .      | 33 |
| 4.4 | Parameter Validation: Generated Test Case. . . . .                                 | 37 |
| 4.3 | Agent Testing Techniques. . . . .                                                  | 37 |
| 4.5 | Security: Generated Test Case. . . . .                                             | 38 |
| 4.6 | KDE of Success Rates in Martin-Lopez et al. (2022). . . . .                        | 39 |

# List of Tables

|     |                                                                     |    |
|-----|---------------------------------------------------------------------|----|
| 4.1 | Impact of Industry Interviews on Development and Outcomes . . . .   | 30 |
| 4.2 | Endpoint and Operation Count in examined APIs. . . . .              | 32 |
| 4.3 | Result comparison of ‘Spec+’. . . . .                               | 35 |
| 4.4 | Parameter Value Coverage Comparison. . . . .                        | 36 |
| 4.5 | API Testing Results. . . . .                                        | 38 |
| 4.6 | Comparison of Fault Codes between Original Spec and Spec+. . . .    | 39 |
| B.1 | Summary of API Testing Practices Across Different Companies . . . . | V  |

# 1

## Introduction

To ensure the quality of software products, it is essential to have comprehensive testing methodologies. One critical aspect of software systems that requires extensive testing is an Application Programming Interface (API). APIs are a software intermediary that allows communication between software applications, for example between a server and the client application. A Representational State Transfer API (REST) is a type of web API that adheres to a set of guidelines and constraints based on the principles of the REST architectural style (Fielding, 2000). APIs that follow this style are often also called RESTful. RESTful APIs have become the de facto industry standard for web applications, enabling data sharing and communication between systems (Golmohammadi et al., 2022). Due to the capabilities and use cases of REST APIs, it is important to have efficient testing procedures in place to guarantee software reliability and quality assurance.

Large language models (LLMs) are sophisticated AI models capable of processing, understanding, and generating human-like text. The LLM’s ability to understand and generate human language makes them potentially suitable for tasks such as interpreting API documentation, formulating test cases, and detecting potential issues (Radford et al., 2018).

Although LLMs are powerful as single agents, they face limitations when handling complex tasks that require multiple steps of reasoning or diverse specialized instructions. Multi-agent systems (MAS) address this limitation by enabling multiple agents to work together, each handling specific aspects of a complex problem. By combining multiple LLMs in a multi-agent setup, complex tasks can be broken down into manageable subtasks, with each agent specializing in different aspects while working towards a common goal (Guo et al., 2024).

This thesis aims to explore the utilization of LLM MAS to automate the generation of test cases from OpenAPI specifications (OAS) (OpenAPI, 2021). Software testing is not a monolithic task, but rather comprises of a multitude of smaller interrelated subtasks. This makes MAS a suitable approach, as the diverse subtasks can be assigned to different agents. Additionally, we introduce ‘Spec+’, a method to enhance API specifications using the knowledge embedded in LLMs. The goal is to investigate whether LLMs can help in the development and testing of REST APIs.

```
1  {
2    "name": "Get new album releases for a specific
      country (Sweden)",
3    "request": {
4      "method": "GET",
5      "url": "https://api.spotify.com/v1/browse
      /new-releases?country=SE"
6    },
7    "expected_response_code": 200
8  }
```

Figure 1.1: Example of an API test case.

## 1.1 Problem Formulation

Figure 1.1 illustrates a sample test case for a REST API endpoint. This example shows a GET request to the Spotify API to fetch new album releases in Sweden. The test case specifies the HTTP method, GET, the request URL, as well as the expected response code (200). Structured test cases, such as this one, are important to verify that the API is behaving as expected. There is a variety of automated testing methodologies that help with the generation and execution of such test cases, supporting developers in this way and enhancing productivity.

Traditional automated testing approaches often face challenges in effectively capturing the complexities of REST API specifications. API documentation is meant to be comprehensive and human-readable. However, in reality they tend to be incomplete and difficult to understand (Golmohammadi et al., 2022). This often leads to potentially undetected defects. In addition, it can be a laborious and error-prone process for developers to manually translate the documentation into test cases, which makes it more difficult to quickly modify tests in response to modifications to the API or its specifications (Ed-Douibi et al., 2018).

LLMs could be a viable alternative to traditional automated testing methods, potentially addressing above mentioned shortcomings (Radford et al., 2018). Tasks such as analyzing API documentation and creating test cases could be solved using LLMs due to their ability to comprehend natural language. The time spent on testing could be reduced if these tasks are automated (Wang et al., 2024).

## 1.2 Goals

The overarching goal of this thesis is to develop a proof of concept for an end-to-end automated REST API testing framework leveraging LLMs, with the goal of providing insights into LLMs' comprehension of REST API specification. These models have a capacity for reasoning that could improve the interpretability of the automated testing process. Thus, providing insight into the reasoning behind individual test cases and their results (Bang et al., 2023; Wu et al., 2023).

The first objective is to investigate the current API testing process in the industry. The objective is to understand which parts of the testing process are currently automated or manual and where an agent-based approach can be implemented to alleviate manual labor, allowing developers to focus on other tasks. This will be done by conducting a short survey with industry professionals from various sectors.

The second objective is to define a process for automated API testing using a MAS. The inclusion of LLMs in the testing framework aims to leverage their knowledge base to assess their ability to generate an effective test suite. Using the understanding and processing capacity of LLMs, the framework can potentially overcome the limitations in current automated testing tools without being developer-hour intensive (Zhang and Arcuri, 2023).

Three research questions have been formulated that will be the main focus of this thesis project.

1. (a) What does a typical testing process for REST APIs look like in the industry?  
(b) What parts of the process are currently automated and to what extent?
2. (a) Is it feasible to utilize an LLM to enrich API specifications, named Spec+, by guiding it at a higher level compared to existing methodologies, and without relying on external knowledge or predefined rule-based prompting?  
(b) What value can the ‘Spec+’ document provide to human consumers and automated systems?
3. How can a multi-agent system architecture be designed for generating test cases directly from API specifications?

Addressing the first research question to a comprehensive extent requires a thorough investigation that exceeds the scope of this project. Therefore, this question will be approached as a case study using interviews with accessible industry practitioners. For the second question, it is important to acknowledge that instructions for completing any given task can be supplied on a wide variety of levels, ranging from very detailed to high-level guidance. This research aims to examine the feasibility of providing LLM with high-level guidance with the goal of evaluating its ability to autonomously work with API specifications. The third research question seeks to explore the implementation of a MAS architecture for generating test cases directly from API specifications, evaluating in this way the potential advantages of such an approach.

The main focus of the study is to develop and assess the multi-agent system for REST API testing. As part of our qualitative research component, we conduct interviews as an auxiliary element to provide real-world insights and information that helps guide the development of the system. Their goal is to ensure that the system addresses actual problems faced by practitioners in the field. By incorporating these insights, this study aimed to develop a system that is not only theoretically sound but also practically relevant to industry practices. The interviews thus serve as a foundation

for the research, helping prioritize functionalities that are most valuable in real-world API testing scenarios.

# 2

## Background

This chapter will be divided into two parts. First, the theoretical background will be explored, including an introduction to REST APIs, software testing, and challenges in REST API testing. The theoretical background will also include an introduction to LLMs and how they have been used in software testing and in Multi-Agent Systems. The last part of the background chapter will present relevant work from the past years.

### 2.1 Theoretical Background

#### 2.1.1 RESTful APIs

Representational State Transfer APIs are fundamental in web development. REST is not a framework, but rather a set of guidelines for the creation of APIs. RESTful APIs allow for stateless communication between client and server throughout the World Wide Web by using standardized protocols. One of those protocols is HTTP and it includes operations such as GET, POST, PUT and DELETE (Fielding, 2000).

The REST principles offer a foundation for the creation and advancement of web services. The focal point of the REST architecture is the stateless nature of RESTful APIs. The statelessness ensures that each client request contains all the required information for the server to understand and process the request. This enables the use in distributed systems and facilitates a division of responsibilities between client and server implementations.

Another important aspect of REST APIs is the uniform interface with a set of predefined operations for all requests. It ensures that all interactions between the client and the server are conducted in a consistent manner. This simplifies the architecture, making it easier for systems to communicate and interact with each other (Fielding, 2000).

The OpenAPI specifications (OAS) offers a robust framework for describing and documenting RESTful APIs. Developed to promote clarity and compatibility between services, the OAS provides a standardized format for API documentation that is both human-readable and machine parsable. This standardization has enabled developers to design, develop and document APIs with a high degree of consistency and efficiency (Golmohammadi et al., 2022). By defining a common interface, OAS

facilitates the use of automated tools for API development and documentation. This standardizes development and improves usability across implementations on different platforms (OpenAPI, 2021).

The OAS is often represented as a JSON or YAML document. It includes main keys such as:

- **openapi**: Specifies the version of the OAS used (e.g. “3.0.0”).
- **info**: Provides metadata about the API, such as:
  - title: The name of the API.
  - description: A brief description of the API.
  - version: The version of the API (Important to note that the version of the API is not the same as the version of the OAS).
- **servers**: An array of Server objects, which provide information on the connection to each server. Each server object must include the base URL for the API.
- **paths**: Defines the available paths (endpoints) and operations for the API. Each path object usually includes:
  - summary: A brief summary of the endpoint’s purpose.
  - parameters: A list of parameters accepted by the endpoint.
  - responses: The expected response from the endpoint, including:
    - \* description: A description of the response.
    - \* content: The data type and schema of the response.
- **components**: Various schemas for the API, including potential definitions, security schemes, and different parameter templates.

These keys are part of fixed fields defined by OpenAPI and define the base structured and behaviour of the API in question. OpenAPI often uses the PetStore server and specification (Figure 2.1) as an example of OAS and its use.

Cacheable responses in REST architecture are used to improve network efficiency and improve user experience by allowing data to be reused for identical requests, thus reducing the need for repeated requests. This mechanism uses the HTTP headers present to decide cache behaviour (Fielding, 2000). Cacheability is an important aspect of REST APIs that is often tested by developers.

Lastly, the layered system design principle in RESTful architecture allows for the construction of applications that can be organized into hierarchical layers, according to a functionality. The system can therefore be scaled by adding or modifying layers without affecting the entire architecture (Fielding, 2000).

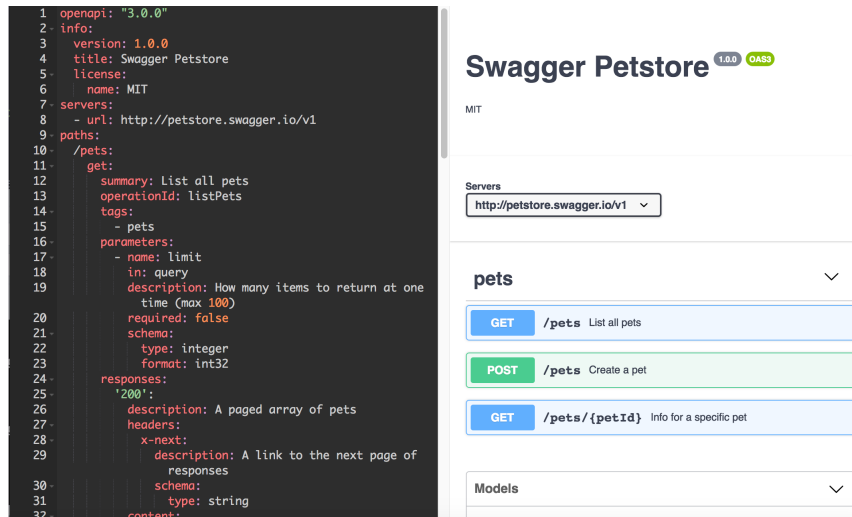


Figure 2.1: Example of OpenAPI Specification (OpenAPI, 2021).

### 2.1.2 Testing Methodologies in Software Development

One of the most important tasks in software development is quality assurance and reliability. Therefore, software often goes through numerous iterations of testing to ensure its robustness. There are a number of different methodologies. Different methodologies will test different aspects of the software. This section compares the main testing paradigms.

Software testing methodologies can be categorized into two paradigms: white-box testing and black-box testing. These paradigms differ on the level of access the tester has to the software’s code (Myers et al., 2012; Khan and Khan, 2012). In this thesis, black-box testing is employed due to inaccessibility to the underlying source code of the web service under investigation. This approach allows for the assessment of the API only from an external point of view. Since this thesis will conduct testing following a black-box methodology, it will be covered in more detail below along with its use in REST API testing.

White-box testing is applicable to different levels. It can be applied to unit testing, the lowest level, which is the testing of a small piece of code. The next level is integration testing, which examines how different parts of the code interact. And on the highest level, there is system testing which is examining how the system as a whole is performing. The foundation of white-box testing can be traced back to McCabe (1976), who introduced code coverage as a metric to quantify the complexity of software development. Code coverage measures the extent to which the source code of a system has been executed during the testing process. White-box testing requires understanding and access to the internal code and architecture of the software. This methodology is based on the notion that having comprehension of the software’s code can reveal errors and vulnerabilities that may not be apparent to more higher-level testing methods.

In black-box testing, software is considered a “black box” entity with unknown internal components and focuses solely on the output behavior of the system. It

evaluates the output of the software given a specific input. Conducting black-box testing does not require in-depth knowledge about the executed code, which is often the case when utilizing external libraries or when making calls to a remote server. Black-box testing overlooks the internal structure of software, enabling a testing approach that can be applied across various software fields and is independent of the programming languages or technologies used (Martin-Lopez et al., 2021a).

The evaluation process entails verifying the system’s compliance against predefined requirements. This verification is achieved through the formulation and execution of test cases derived from external sources, such as technical specifications. This approach emphasizes the examination of inputs against the corresponding outputs (Myers et al., 2012).

Within the realm of REST API testing, the black-box methodology enables testers to concentrate on the API’s response behaviors to various requests, thus ensuring compliance with the OAS. Black-box testing allows one to evaluate the APIs’ operational efficiency, status code responses, and overall performance metrics without being concerned of the details or inner workings of the source code (Myers et al., 2012; Martin-Lopez et al., 2021a).

### 2.1.3 Challenges in REST API Testing

Studies in RESTful API testing call attention to several key challenges in generating effective unit tests. Ehsan et al. (2022) and Golmohammadi et al. (2022) outline 4 main challenge categories: Security concerns, non-compliance to specifications, not up-to-date specifications, and complex input types.

Security might be a challenge in API testing due to the need to protect sensitive data and ensure authorized access while performing tests. The verification of authentication mechanisms and encryption protocols requires many steps and may require advanced testing strategies to complete. When performing black-box tests, this poses an additional challenge as the tester might have to authenticate multiple times before making API calls (Isha et al., 2018; Coblenz et al., 2023). This complexity underscores the importance of integrating security considerations into the API testing process to maintain system robustness (Ehsan et al., 2022).

Maintaining up-to-date API documentation in parallel with software development is labor intensive. Inconsistencies between documentation and implementation introduced by the human factor together with the number of different documentation standards create a complex environment. This environment requires tools that can handle diverse and incomplete input (Isha et al., 2018; Ehsan et al., 2022).

Lastly, the complexity of data types in API inputs presents a challenge in API testing. The complexity increases as modern APIs often handle a wide range of data types, from simple strings and integers to complex nested objects and file uploads (Ehsan et al., 2022). This diversity requires advanced testing frameworks capable of accurately generating and validating a multitude of data formats. Ensuring the correct handling of these complex data types is a key to the reliability of the API. However, it complicates the process of automating the generation and execution of

test cases (Chen et al., 2020). Explanations and definitions of complex data types are often conveyed through comments or natural language within the specifications. Current automated testing tools struggle to understand natural language. This leads to challenges in generating accurate values for these complex data types during testing processes (Kim et al., 2022).

### 2.1.4 Large Language Models

The paper *Attention is All You Need* by Vaswani et al. (2017) has been central to the rise of LLMs. The authors introduced the basis for LLMs, the transformer. The transformer is a neural network model that utilizes an encoder-decoder architecture with the addition of the attention mechanism. The encoder-decoder architecture consists of two main components: an encoder that processes the input sequence and creates a vector representation of it and a decoder that uses this representation to generate the output sequence. However, in modern LLMs the decoder-only architecture is especially popular. The attention mechanism has enabled a leap in NLP capabilities, allowing models to process words in relation to all other words in a sentence, thereby enhancing understanding and generation of text. It also allows for parallel processing of sequences, giving the model the ability to capture complex dependencies within the data.

Attention can be calculated as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (2.1)$$

where:

- $Q$  stands for the *queries*, each a vector that the attention mechanism aims to find relevant information for.
- $K$  represents the *keys*, a set of vectors that the queries are matched against.
- $V$  denotes the *values*, a set of vectors that are retrieved and weighted according to the matching result of queries and keys.
- $d_k$  is the dimensionality of the key vectors. It is used as the denominator of the scaling factor to avoid overly large values of the dot products, especially in environments with high-dimensional data. This scaling helps the softmax function's gradients remain in a relatively low range.
- The operation  $QK^T$  involves computing the dot product between each query and all keys, resulting in a matrix that captures the matching scores between them.
- The *softmax* function is applied to these scores, converting them into probabilities that sum to 1. These probabilities essentially determine how much each value is emphasized in the output.

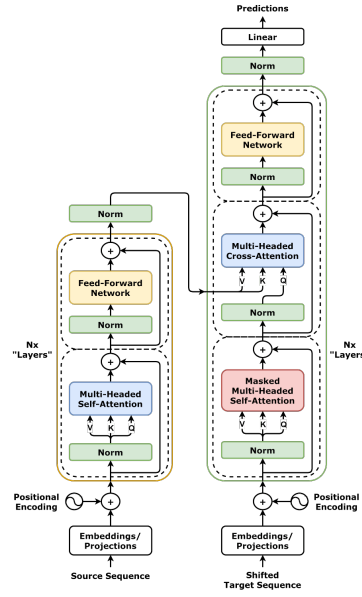


Figure 2.2: The Transformer architecture (Vaswani et al., 2017)

In this context, each vector represents a set of features or attributes of a single token in the input data. A token is the smallest unit of text, such as a word or subword, that the model processes. These features could be learned embeddings that encode semantic and contextual information about words or subwords in a sequence. These vectors help the model decide how important each piece of information is by giving more weight to the relevant details. In this way, the model can better understand the interrelations within the input data.

The output of the Attention function is a weighted sum of the values, where the weights are determined by the softmax-normalized scores obtained from the query-key dot product matches. This mechanism allows the model to dynamically focus on different parts of the input data, which is meant for tasks such as sequence modeling and translation.

The complexity of the attention mechanism is further highlighted by three key aspects:

1. The resulting matrix  $QK^T$  has a size of  $n \times n$ , leading to a memory requirement that scales quadratically with the sequence length  $n$ . This can act like a bottleneck for long sequences.
2. Although the memory scales quadratically, the computational complexity itself involves multiplying matrices of sizes  $n \times d_k$  and  $d_k \times n$ . This results in a computational cost of  $O(n^2 d_k)$ , which means the computational requirements are even more significant for long sequences and large embedding dimensions (Vaswani et al., 2017).
3. The non-linear softmax function: Without the softmax function, the attention operation could be simplified to  $Q(K^T V)$ , which would be much more computationally efficient. However, the softmax introduces a crucial non-linearity

that allows the model to learn more complex relationships between tokens. It normalizes the attention weights, ensuring they sum to 1 and creating a probability distribution over the input sequence.

These factors contribute to the computational intensity of the attention mechanism, especially as sequence lengths increase. However, they are also key to the mechanism’s effectiveness in capturing complex dependencies within the data.

The architecture is built on an encoder-decoder paradigm, which enables the model to understand sequences in a global context by processing the entire sequence of data before generating an output. The encoder processes the input sequence and creates a comprehensive representation of the data. The decoder then uses this representation to generate an output sequence, ensuring that each part of the output takes into account the entire input context. The combination of this architecture together with the implementation of the attention mechanism is what allows the model to focus on relevant parts of the input sequence when generating the output. The ability to selectively focus is why calculating attention is so computationally expensive, as it involves computing attention weights for every pair of positions in the input sequence. This has served as the cornerstone for the development of LLMs, facilitating advancements in NLP tasks such as translation, text generation, and semantic analysis (Vaswani et al., 2017).

A key component of LLMs is their extensive knowledge base, which is built during their pretraining process. This pretraining enables the models to learn patterns, facts, and general knowledge, which can then be applied to various NLP tasks. LLMs, such as the Generative Pretrained Transformer (GPT) series, have since their inception revolutionized NLP tasks. Originally designed to understand, generate and translate natural language, these LLMs have evolved through multiple iterations, going beyond simple text processing to fields that require nuanced understanding (Brown et al., 2020).

## **Tool-Augmented LLMs**

The scalability of LLMs has been a key factor in achieving enhanced performance across a multitude of tasks. However, the limitation of LLMs in not having access to real-time or private data poses a considerable challenge to their applicability to real-world tasks. Addressing these limitations, Parisi et al. (2022) introduces the concept of Tool Augmented Language Models (TALM), a concept that integrates LLMs with external tools to expand their capabilities.

TALM’s core innovation is its method of augmenting LLMs with a text-to-text interface, enabling the integration of tool outputs into the language generation process. A tool refers to any external, nondifferentiable system that the LLM can invoke to perform specific functions that are not possible within its own computational framework. Nondifferentiable systems are those that do not support gradient-based optimization, which is essential for the standard training of neural networks using backpropagation. Examples include retrieval systems, mathematical solvers, or external databases whose operations (such as search algorithms or computational

functions) cannot be directly optimized through gradient descent. These tools can provide access to real-time data, perform complex computations, or interact with databases. Some examples of tools are the *PythonREPL* tool, which allows a model to run arbitrary code (LangChain, 2023c) or the *GoogleSearch* tool, which gives a model access to real-time data (LangChain, 2023b).

The nondifferentiable nature of tools requires different approaches to allow models how to use them. TALM’s iterative *self-play* approach starts with a few examples of how to use the tools in its disposal. By repeatedly practicing with these tools, TALM gets better at using them, which iteratively improves its performance on tasks. TALM has been tested on the Natural Questions (NQ) and MathQA datasets for both knowledge and reasoning, where it showed improved performance compared to non-augmented LLMs. TALM’s ability to outperform larger models without tool augmentation demonstrated its potential to reduce the reliance on model scale for performance gains (Parisi et al., 2022).

### 2.1.5 Multi-Agent Systems

An agent is defined as an autonomous computational entity with the ability to perceive its environment, make decisions and take actions to achieve specific goals (Guo et al., 2024). For the purposes of this study, we focus on Intelligent Generative Agents or IGAS, which are agents powered by LLMs. These agents can process information, generate responses, and interact with other agents (Talebirad and Nadiri, 2023). We focus on IGAS due to their advanced capabilities in processing complex information and generate human-like content, which is particularly relevant for the REST API testing tasks discussed in this study.

A Multi-Agent System (MAS) is defined as a system where multiple agents, each equipped with distinct profiles, strategies, and behaviors, collaborate to solve complex tasks. These agents interact with each other and their environment, engaging in communication and collective efforts (Wu et al., 2023; Topsakal and Akinici, 2023). Unlike single-agent systems, which focus on the internal mechanics and interaction of a single agent with its external environment, MAS emphasize various diverse agent profiles and their interactions to solve problems collaboratively (Guo et al., 2024). The essence of MAS lies in their ability to work towards complex tasks by enabling multiple agents to collaborate. Each agent is powered by an LLM that could have access to tools. Multi-agent conversation plays a key role in the integration of LLMs for tasks of variable complexity. The system’s capabilities are improved by breaking down large tasks into smaller, more manageable tasks, allowing specialized agents to be able to handle subtasks in an effective manner (Li et al., 2024).

REST API testing can be a complex undertaking, with a number of smaller tasks that need to be completed in sequence. MAS are particularly suited for this type of problems due to their ability to segment complex tasks into smaller more manageable components. This segmentation can help by assigning different parts of the testing process to specific agents, each equipped with capabilities targeted toward handling the assigned task. Furthermore, by leveraging multiple agents, a MAS can overcome the nondeterminism of individual LLMs, as collaborative efforts of agents

provide a more comprehensive approach to identify and perform individual subtasks (Liang et al., 2023). This decomposition of tasks not only ensures a more robust task performance, but also prevents individual LLMs from missing context and failing to address the specific aspects of the task accurately. It is a phenomenon called attention dilution, and refers to the model’s difficulty in maintaining focus on relevant information when processing longer inputs (Tamkin et al., 2021). Attention dilution is one of the many reasons LLMs are accompanied by context limits (often referred to as “context window” or “context length”), meaning they are only allowed to a certain number of tokens in a single input. Lastly, the use of MAS enables the development of a system based on various MAS architectures that fit the needs of the task at hand. Examples of MAS architectures are sequential, hierarchical (Wu et al., 2023), or Reflection architectures (Shinn et al., 2023). This approach enhances the system scalability and explainability, as agents can be added or modified, and their interactions can be traced through their communication (Zhuge et al., 2023).

## 2.2 Related Work

### 2.2.1 Empirical Studies on REST API Design and Testing Practices

The challenges and practices in the design and testing of REST APIs have been explored in various studies, showing the gap between theoretical guidelines and practical implementation. Coblenz et al. (2023) conducted a qualitative study to investigate REST API design and specification practices. They used semi-structured interviews with ten experienced developers to identify common challenges, adherence to guidelines, and potential areas for improvement. Their findings showcase the complexities in REST API development, such as difficulties with authentication and authorization, the commonness of outdated or vague documentation, and the lack of standardization in error handling mechanisms. These challenges often result in inefficient workflows and suboptimal API usability and testing.

The study revealed that some design guidelines, such as ensuring statelessness and using JSON for data transfer, are widely adopted. However, others, such as the hypertext-driven approach, are often considered impractical by practitioners. Additionally, the reliance on manual testing due to the complexity of automated tools aligns with industry trends where time and resources limit the adoption of advanced testing techniques.

These findings are relevant to the interview component of this thesis, which explores developer perspectives on REST API testing practices and tool usability. Similar to the challenges identified in the Coblenz et al. (2023) study, this work aims to understand how practitioners perceive existing testing approaches and the extent to which tools can alleviate these issues.

## 2.2.2 Rest API testing with LLMs

### RestGPT

Song et al. (2023) introduce a framework that uses the capabilities of LLMs to interact with and test RESTful APIs, called RestGPT. This framework utilizes a novel approach by integrating natural language understanding with the technical specificity required for API interactions. RestGPT connects the complex natural language queries and the structured world of API requests. Hence, it offers a more intuitive way for developers and testers to interact with APIs. This is achieved through an architecture consisting of three agents:

- Planner, whose task is to decompose the user’s instructions, converting them into manageable sub-tasks.
- API Selector, who maps the sub-tasks to actionable queries and their corresponding API calls.
- Executor, who is invoking APIs and is parsing their responses according to the user’s initial query.

This approach enhances the interaction between LLMs and APIs and improves the efficiency and intuitiveness with which developers and testers can engage with APIs. RestGPT presents a leap forward in REST API testing, offering a more natural, intuitive means for performing API interactions and evaluations (Song et al., 2023).

### RESTGPT: Echoing Innovations

Despite sharing a similar name with Song et al. (2023)’s RestGPT, RESTGPT by Kim et al. (2024) represents a distinct approach with different objectives. While RestGPT focuses on multi-agent interaction with APIs, RESTGPT introduces a methodology for leveraging LLMs to improve REST API testing through rule extraction from OAS. The technique revolves around extracting machine-interpretable rules and generating example parameters directly from the natural language found in the OAS. Unlike traditional methods that mostly rely on the structure data part of the OAS, RESTGPT takes advantage of the intrinsic context-awareness of LLMs to parse and interpret the unstructured information within the OAS. This approach is aimed towards the augmentation of the original OAS with the generated rules and values, offering a more comprehensive testing framework. RESTGPT improved the precision of rule extraction and parameter value generation compared to NLP2REST (Kim et al., 2023), the previous work of the authors, as well as ARTE (Alonso et al., 2023).

However, the study by Kim et al. (2024) presents certain limitations, namely the current implementation of RESTGPT is limited to detecting faults that manifest as 500 server response codes (Kim et al., 2024). This limitation, while narrowing the scope of the study, is omitting a range of bugs potentially related to CRUD operations, which could be significant in practical applications.

Although not explicitly stated in the paper, we can make the assumption that the

focus on handling only 500 server response codes might be due to the authors' plans to extend their implementation in the future. This could suggest that the low-level prompting is very specific to their current implementation, which is centered around detecting 500 server responses.

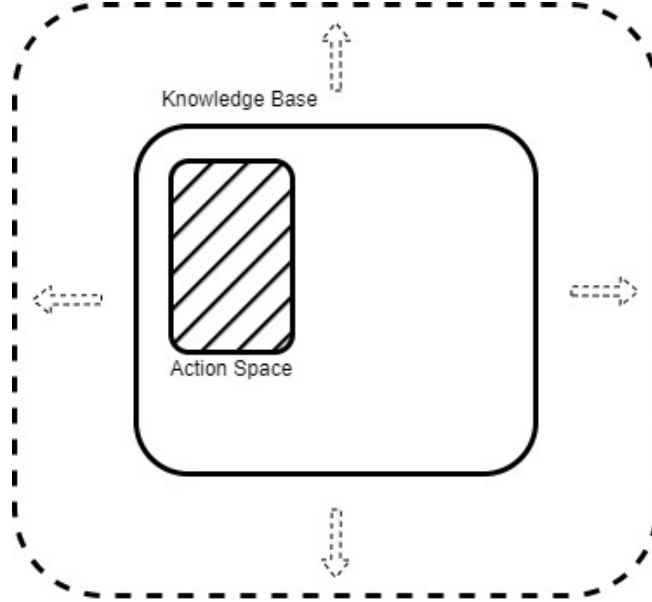


Figure 2.3: Action Space in the LLM knowledge base

Although the goal is similar, to enhance specifications using LLMs, the approach in this thesis is different. Kim et al. (2024) use a Rule Generator to extract four predefined rules regarding operational constraints, parameter constraints, parameter type and format, and parameter examples. This is done by providing the model a set of crafted prompts with the goal of generating parameter values. Our approach is to give the LLM a high-level target to work towards as well as guidelines as to what to avoid. This allows the model the freedom to make decisions about how to approach the task and what enhancements to recommend.

Rule-based systems, while effective in certain limited contexts, face limitations when dealing with more dynamic or complex scenarios. Their reliance on predefined, rigid rules restricts their ability to adapt to new unforeseen circumstances without manual intervention. This rigidity becomes particularly apparent in contexts like API testing, where unexpected errors or new operational scenarios may arise that the system is not equipped to handle. Moreover, as systems scale and the number of rules increases, managing and maintaining the rules becomes cumbersome, potentially leading to inefficiencies or errors. As Yoon et al. (1994) note, the static nature of rule-based systems limits their scalability and adaptability, preventing them from efficiently handling complex or unstructured data. Kanervisto et al. (2020) further argue that such systems are constrained by their 'action space,' meaning that they can only perform actions within the confines of their predefined rules, thus limiting their ability to generalize and expand beyond initial configurations.

By providing the model with more flexibility, we aim to expand the action space and allow the LLM to potentially access more of the knowledge base while also giving

the system potential for scalability with future models (Figure 2.3). The argument behind this is that models have access to their current knowledge base, which will only expand with future versions or newer models. The flexibility in our approach promotes an adaptive interaction with the OAS and is not limited to specific error codes, allowing the agent to address CRUD operations.

One could, however, argue that these two approaches have a trade-off of accuracy vs. scalability. Although the benefits of a scalable system can be proven beneficial, providing the model with more precise instructions could counteract the inherent nondeterministic nature of LLMs. These two approaches complement each other in the trade-off spectrum. The low-level, rule-based method proposed by Kim et al. (2024) focuses on a narrow set of tasks with greater precision. This specialized approach is ideal for cases where precision in a well-defined task is of great importance, ensuring that the model performs with minimal error. However, as a consequence of this high precision, the system is less adaptable and does not easily scale to a wider range of tasks or more complex scenarios. In contrast, the high-level approach in this thesis favors scalability and holisticness over precision. It enables the system to handle various scenarios across an entire API specification, providing an adaptable framework for testing. However, this flexibility may result in lower precision, particularly for highly specialized tasks that have strict constraints. In a sense, the two approaches complement each other along the trade-off spectrum: while the low-level system performs better in accuracy for defined tasks, the high-level system offers the potential for broader applicability and future scalability.

# 3

## Methods

### 3.1 Approach

This section presents the methodologies used to answer previously defined research questions. It is split into two segments, ‘Industry Insights’ and ‘Leveraging Multi-Agent Systems for REST API Testing’.

The ‘Industry Insights’ section will explain the approach to answering the initial research questions, focusing on current API testing processes and the balance between automated and manual tasks. It is done through a qualitative approach consisting of semi-structured interviews with industry professionals. This is followed by the ‘Leveraging Multi-Agent Systems for REST API Testing’ section, which will explore the engineering and design choices made to implement the MAS for API testing.

#### 3.1.1 Industry Insights

To assess the existing challenges in the testing process, this study conducts qualitative interviews with professionals from industries such as industrial communication, automotive, and web development.<sup>1</sup> The selection of candidates from diverse companies was mainly based on availability and practicality, rather than extensive scientific sampling methods. Participants were asked to make a self-assessment and position themselves along two positions: the size of the company and how software-oriented the company is (Figure 3.1). This subjective assessment gives valuable insight into the testing processes and challenges in diverse organizations.

The objective of this methodology is to improve the understanding of the testing landscape by receiving insights into testing processes and challenges in the industry. It is important to note that this approach is based on subjective placement of the companies by the participants. Therefore, offering a qualitative perspective, rather than quantitative.

The interviews are semi-structured to facilitate a conversational environment with the goal of getting insight on the development and testing process for REST APIs in the industry. This methodology also aligns with recent research in REST API

---

<sup>1</sup>Given the for-profit nature of the participating organizations, anonymizing the interviews was essential to protect company interests. Participants were explicitly informed about this anonymity requirement and the study objectives, and provided their consent accordingly, ensuring both ethical standards and company confidentiality were maintained.

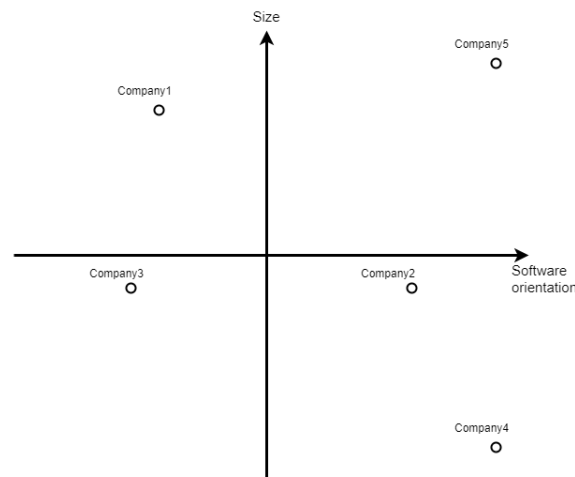


Figure 3.1: Dimensions of interviewed companies

development and testing (Coblenz et al., 2023). Semi-structured interviews are a flexible and effective qualitative research method (Esaiasson, 2007; Hove and Anda, 2005). Qualitative research methodologies have traditionally been extensively used in fields such as sociology and business (Lee and Liebenau, 1997; Esaiasson, 2007). However, their role as a valuable complement to quantitative approaches has also been increasingly recognized in more mathematically rigorous and scientifically oriented disciplines, including computer science (Hazzan et al., 2006) and software engineering (Hove and Anda, 2005).

Semi-structured interviews are guided by a set of predetermined questions or topics and allow for open-ended responses and questions based on the interviewee’s answers. Semi-structured approaches give interviewers the flexibility to explore new topics that arise during the conversation, offering a deeper understanding of the subject (Esaiasson, 2007). Although there is a guide or framework, the interviewer has the freedom to adapt the questions to the flow of the conversation. This method allows for responses that capture the nuances of the interviewee’s experiences and viewpoints (Hove and Anda, 2005).

The preparation for the interviews entails the development of a guide with key topics and questions. They are meant to explain the purpose of the interview to the participants (Esaiasson, 2007). For the purposes of this study, the RQ1 acted as a guide for interviews (all the questions that surfaced during the interview can be found in the Appendix A). The RQ was shared with the interviewees prior to the interview, allowing them to prepare for the interview themes:

1. What does the testing process for REST APIs look like in your company?
2. What parts of the testing process are currently automated and to what extent?

## Data Capturing & Collection

Data capturing has been primarily done through note-taking. Due to the constraints of conducting interviews independently and within a limited time frame, a dedicated note-taker was not feasible. This method was selected to avoid inhibiting the candidness of the interviewees with recording devices. However, this approach carries some inherent risks, such as the note-taker's selective attention and judgment about what is important, and the inability to listen back to the interviews (Esaiasson, 2007).

## Interview Process Outline

Step 1: Pre-Interview Preparation (7-5 Days before the interview)

- **Action:** The interviewee is emailed a formal invitation to participate in the study.
- **Details:** The email includes a brief introduction to the study.
- **Outcome:** The interviewee reviews and consents to participate in the study.

Step 2: Question Sharing (5-4 Days before the interview)

- **Action:** The research question is shared with the interviewee, along with further details insights expected from the interview.
- **Details:** These cover key themes related to the testing process of REST APIs (e.g., testing methods, automation).
- **Outcome:** The interviewee has time to prepare responses, making the interview more focused.

Step 3: Interview Scheduling (4-2 Days before the interview)

- **Action:** Interview date and time are finalized, and a virtual meeting link is shared.
- **Details:** The interview is scheduled based on mutual availability, and interview duration (e.g., 45-60 minutes) is communicated.
- **Outcome:** Interviewee confirms participation and receives a calendar invite.

Step 4: Interview Day

- **Action:** Semi-structured interview is conducted via video call.
- **Details:** The interview begins with a brief reintroduction to the study objectives, followed by open-ended and follow-up questions as necessary.
- **Outcome:** Interviewer takes notes on the interviewees responses.

Step 5: Data Analysis and Reporting (Within 1 week post-interview)

- **Action:** The collected interview notes are reviewed and analyzed.
- **Details:** The interviewer identifies key insights, challenges, and patterns in the interview data.

- **Outcome:** The results are synthesized into a report on the study’s findings on the REST API testing processes.

### 3.1.2 Leveraging Multi-Agent Systems for REST API Testing

This section explores the application of an LLM-based multi-agent approach for automating testing tasks. This research explores the integration of LLM-powered agents within the software testing domain, drawing inspiration from the AutoGen framework (Wu et al., 2023).

As demonstrated by previous research discussed earlier in this thesis (see 2.2.2), agents powered by the knowledge base of LLMs have shown the capability to further enrich API specifications. A vast array of OAS are available online, suggesting that LLMs have likely seen numerous API specifications during their training. This exposure enables them to generate suggestions based on patterns observed in various API specifications, which may include widely recognized best practices (Brown et al., 2020; Radford et al., 2018).

Furthermore, the use of a MAS for testing REST APIs is a particularly well-suited solution, compared to a single-agent LLM, due to the complex and multifaceted nature. REST API testing includes a wide range of tasks, including analyzing specifications, identifying testing needs, generating test cases, running tests, and even analyzing test results, each requiring different expertise and approaches. A MAS allows for the decomposition of these complex testing tasks into smaller, more manageable components, with each agent specializing in a specific aspect of the testing process. This modular approach improves flexibility and scalability, allowing the system to adapt to different API structures and testing requirements. Furthermore, the collaborative nature of MAS enables agents to share information and insights, potentially uncovering subtle interactions or edge cases that might be missed by a single, monolithic approach. Using the capabilities of multiple agents, a MAS can provide a comprehensive testing strategy that addresses the inherent challenges of REST API testing. By defining the subtasks involved in REST API testing and demonstrating how MAS can handle these tasks effectively, we provide a stronger justification for the applicability of MAS in this context. The specialized agents within a MAS may improve testing efficiency and adaptability. This approach is supported by recent literature, including Zhuge et al. (2023) and the concept of natural language-based societies of mind, further validating the suitability of MAS for REST API testing.

### 3.1.3 Implementation Details & LangGraph

#### Development Process and Prompt Engineering Challenges

A significant portion of the development time for this project was dedicated to prompt engineering and getting the framework to function as intended. The process of crafting effective prompts for the agents, ensuring that they understood their roles

and tasks within the system, as well as fine-tuning their interactions was a complex and time-consuming process.

This extensive effort in prompt engineering was important for the successful implementation of the MAS. Furthermore, highlights the challenges and nuances inherent in developing LLM-based systems. The iterative nature of this process involved numerous adjustments and refinements to achieve the desired agent behaviors and system performance.

### LangGraph Implementation

The LangGraph framework (LangChain, 2024a) was used for the implementation of the MAS. LangGraph is built on the LangChain Expression Language (LCEL). LangChain was initially built for the development of LLM applications, usually in the form of a *Chain* (LangChain, 2023a). It was introduced as an easy way to implement graphs in LLM applications. In these graphs, each node is an agent with a predefined persona, and the edges show which agents it is able to communicate with. This approach enables more complex task completion by the LLM application, as the agents are often required to reason about the current state of the task before deciding on the next step (LangChain, 2024a).

The bases of LangGraph are the *StateGraph* and *AgentExecutors*.<sup>2</sup> *AgentExecutors* are LLM-powered agents that have the ability to call functions based on their given system prompt (persona), also known as tool calling. A system prompt is a message that sets the objectives the agent should accomplish. The system prompts are usually not visible to the user. The *StateGraph* describes the data structure that keeps track of all inter-agent communication. The *StateGraph* can be represented as a Python dictionary, where the keys are the names of the agents in the graph, and values that are arrays of all responses from that particular agent. This is important for agents to be able to execute the tasks allowing access to the knowledge of previous steps in the process (LangChain, 2024a).

The LangGraph graph is built upon the *StateGraph*, where each node is the invocation of the *AgentExecutor*. The invocation of an agents means the agent is activated once visited and produces a response. There are three types of edges in LangGraph: the *starting edge*, *normal edges*, and *conditional edges*.<sup>3</sup> The *starting edge* connects the start of the graph to a particular node, setting the starting node for the process. *Normal edges* connect two nodes in an obligatory manner. This means that two nodes connected to a normal edge will always be visited, one after the other, resembling a chain in LangChain. Lastly, the *conditional edges* are LangGraph’s most notable feature. These edges are used in conjunction with a function that is embedded in the node and is used to make a choice of which node to visit next. An

<sup>2</sup>*StateGraph* and *AgentExecutors* are proprietary Python objects implemented within the framework. These terms are used in their technical context throughout this discussion to maintain consistency with the framework’s architecture.

<sup>3</sup>*Starting edges*, *normal edges*, and *conditional edges* are proprietary Python objects implemented within the framework. These terms are used in their technical context throughout this discussion to maintain consistency with the framework’s architecture.

illustration can be seen in Figure 3.2. Based on the agent’s decision, either a tool is called, or the agent interaction is concluded, and a response is returned to the user. Once the graph is built, it can be compiled and returned as a runnable (LangChain, 2024a).

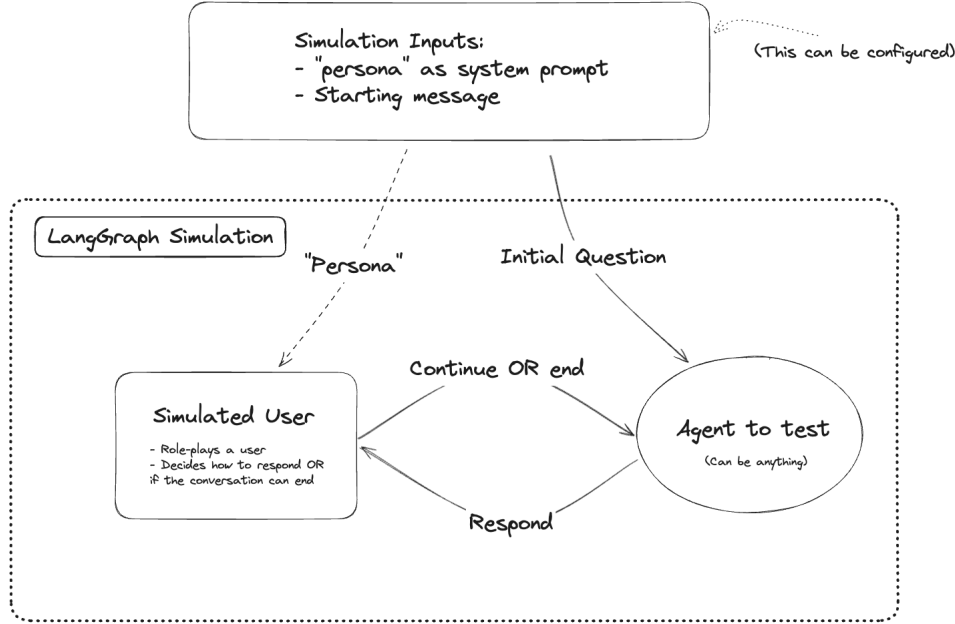


Figure 3.2: Illustration of conditional edge usage (LangChain, 2024b).

### 3.1.4 System Design

In this project, the MAS will be powered by a pretrained LLM. For the purposes of this study, all agents are powered by GPT-3.5 Turbo (Peng et al., 2023) and GPT-4 (OpenAI et al., 2024). This creates a balance between context length and cost, as GPT-4 has a much higher context window (almost 4 times that of GPT-3.5), but also comes at a higher price point (approximately 20 times higher). A modern LLM has received training that provides it with a knowledge base sufficient for taking on a wide variety of tasks. Domain adaptation could offer very marginal benefits, but also carry a risk of worsening the model’s performance by modifying its already finely-tuned weights (Ramponi and Plank, 2020).

Two agent teams, an API Specification Team and an API Testing Team, each with their own goals and capabilities, have been developed.

The API Specification team is led by a supervisor agent whose task is to keep track of the inter-agent conversation in the system and make a decision on which agent must act next. The supervisor has a description of the task for which they are responsible, in the form of a system prompt. The supervisor is also employed with a function that allows it to choose between conditional edges that lead to different agent nodes. Each agent in the team has a description of their designated sub-task

in their system prompt and is instructed to declare the task complete based on their judgement. The API Testing team is built with a different approach, using a basic Reflection architecture (Shinn et al., 2023).

The LLMs have a limited input length based on the model context length. The OAS was therefore segmented and processed by the agent, two endpoints at a time. This is done through a sliding-window approach, where the first set of two endpoints was processed initially, followed by the next two, and so on. This methodology facilitated sequential input processing, albeit with some constraints on the comprehensiveness of each input segment. To support this approach, functions were developed to programmatically extract relevant schemas for the selected endpoints, creating a ‘mini’ specification that included only the necessary elements for each segment. Although this approach is not ideal, as it potentially restricts the LLM’s ability to perceive and analyze interaction between all available endpoints, it was a necessary compromise to advance the project within the current technical constraints.

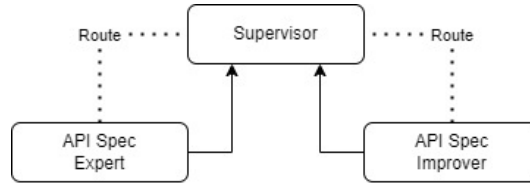


Figure 3.3: Specification Enhancement Team

### API Specification Enhancement team

The API Specification Enhancement team is responsible for using the knowledge base of the model to enhance the API Specification and generate the ‘Spec+’. This team is made up of the supervisor and two additional agents: An API Expert and an API Improver (Figure 3.3). The API Expert is tasked with analyzing the given specification and provide improvement suggestions based on its judgement. To further showcase the inherent benefits of the model’s reasoning capabilities, high-level goals and guidelines are given to the agent, allowing them to make decisions based on their judgement and reason about them. Once the improvements suggestions have been made, the API Expert is reporting back to the supervisor, who judges whether to route the task to the API Improver. The API Improver’s task is not complex, receive the improvement suggesting from the API Expert, and apply them to the specification. In the case of the API Improver, the task itself is not complex, but the length of the API specification is a challenge. Models can reach their output token limit when dealing with lengthy responses. This is due to computational requirements for calculating attention. This could result in the model reaching the token limit before the full specification is generated. This could be mitigated by splitting the task among two agents. So for the sub-task of improvement implementation, the prompting is aimed more towards how the agent should provide the answer to the supervisor, rather than explaining the task. The agent is asked to provide the full answer verbatim and not declare the task complete until the full specification is covered. In this way, if the Improver responds to the supervisor without declaring the task complete, the supervisor reroutes the task back to the Improver to continue.

### API Testing team

This task was not hindered by context length limitations, allowing us to use a more complex architecture to achieve better results. The architecture of the API testing team is based on the self-reflection methodology (Shinn et al., 2023). This architecture includes two agents: an Actor, designated as the API Developer, and an Evaluator, supervising the API Developer. This structure results in an iterative process where the Actor is responsible for generating test cases based on the OAS and the feedback from the Evaluator. Only a segment of two API endpoints are input at a time, as stated earlier. Therefore, the test cases generated are limited to the context of that segment only.

The Evaluator is called once the test cases have been generated. It reviews the test cases produced by the Actor, providing critiques, feedback, and recommendations. This feedback loop is designed to refine the quality of the test suite through user-defined  $N$  iterative cycles of generation and reflection (see Figure 3.4). This cycle ensures that test cases are vetted before being executed outside of the MAS. This way the API’s functionality is tested as thoroughly as possible within the LLM knowledge base.

Agents have complete freedom to decide their course of action on how to approach the generation of test cases. This autonomy allows for a diverse range of testing strategies in test case generation. Although we recognize its relevance, advanced methods such as fuzzing (Martin-Lopez et al., 2021b) are beyond the scope of this thesis. Instead, the project aims to provide a testing methodology while showcasing its autonomy.

It is important to note that the execution of test cases within self-reflection cycles presents significant challenges. In particular, due to the authentication requirements of APIs. Each API may require different authentication methods, some of which can be quite complex. Implementing this would require developing specialized agents or even teams of agents to handle the various authentication steps of APIs. While this is an important aspect of API testing, it falls out of the scope of this thesis, which aims to evaluate the possibility of autonomous API testing rather than providing a full implementation.

Due to these considerations and technical constraints, an alternative solution was employed for the execution phase. Instead of implementing additional agents to run the test cases after the self-reflection iterations, a LangChain *Chain* was implemented.<sup>4</sup> This chain is designed to call a Python function that runs the finalized test cases, analyzes the results, and returns both the results and the feedback to the user. This integration was necessary to ensure that the test cases were run within the scope of the system, while also remaining within the practical limits of this project.

---

<sup>4</sup>Chains refer to sequences of calls - whether to an LLM, a tool, or a data preprocessing step (LangChain, 2023a).

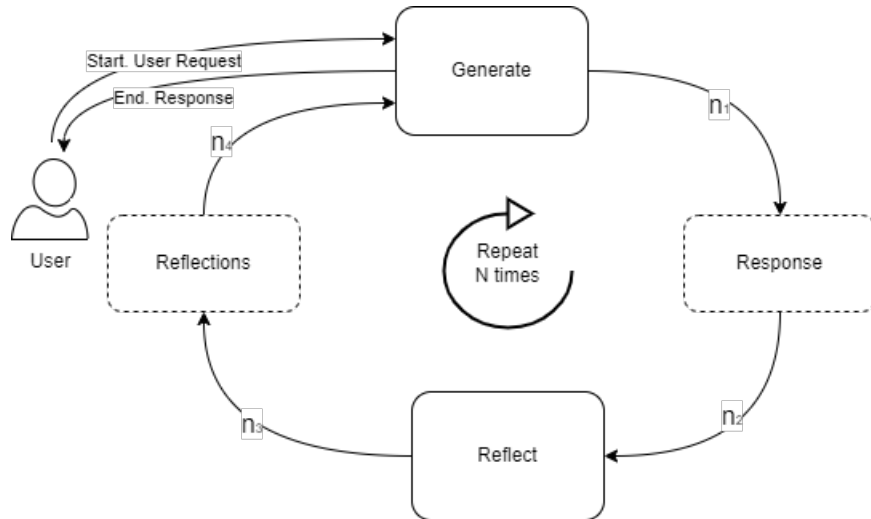


Figure 3.4: API Testing Team

### 3.1.5 Evaluation Setup

The workflow is based on the two teams. There are two flows in the process (see Figure 3.5). In the first flow, the OAS is passed through the Testing Team for test case generation. The cases are then run, and their results are analyzed by the agent. In the end, a summary is presented detailing how many cases passed the test and how many failed. The agent can also provide feedback on the potential reasons for the failed cases. The second flow begins by passing the OAS through Enhancement Team to generate the ‘Spec+’. Then the ‘Spec+’ is following the same process as in the first flow.

The goal is to be able to compare the test results from the two flows to assess whether the ‘Spec+’, and in turn the LLM’s knowledge base, can be used to improve the process and generate better test cases.

To consider the test results of the original OAS as a good baseline, they need to be assessed against an industry standard. Therefore, in this final step, the test results of the Testing Team using the original OAS is compared against the test results of an already established testing automation tool.

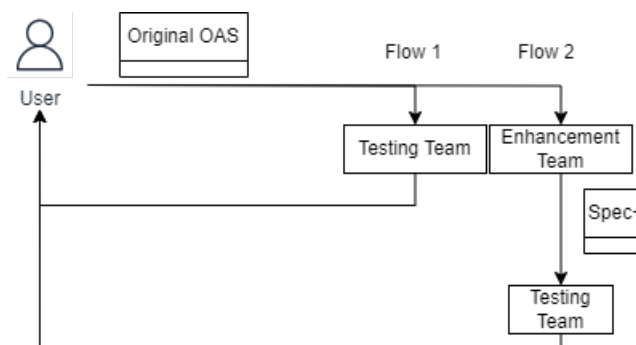


Figure 3.5: API Testing Workflow

## 3.2 Evaluation of the Results

### 3.2.1 Evaluation of ‘Spec+’ Documents

The evaluation of the generated ‘Spec+’ documents presents a significant challenge due to the subjective nature of text generation and the absence of a ground truth data. The original document, while serving as a baseline for improvements, cannot by itself be considered a ground truth since the objective is to enhance and improve on the quality. Therefore, no ‘perfected’ document exists against which the ‘Spec+’ documents can be objectively measured. This lack of a clear standard requires a qualitative approach to assess the effectiveness of the improvements.

A selection of ‘Spec +’ documents have been sampled and examined against their original counterparts. The evaluation focuses on improvements in clarity and completeness, analyzing how ambiguities or omissions in the original documents were addressed. By manually comparing the enriched specifications with the original documents, the goal is to evaluate the improvements in integrating LLM insights.

### 3.2.2 Evaluation of Test Cases

Evaluation is performed by generating two sets of test cases from the MAS: one set from the original OAS and another from the enriched ‘Spec+’. Both sets have been evaluated against the results from a traditional REST API testing framework-RESTest (Martin-Lopez et al., 2021b)- to assess the benefits of the MAS approach. These two sets were then compared to each other to evaluate the enhancement in test quality and coverage.



| TCL | Input criteria  | Output criteria          |
|-----|-----------------|--------------------------|
| 0   |                 |                          |
| 1   | Paths           |                          |
| 2   | Operations      |                          |
| 3   | Content-type    | Content-type             |
| 4   | Parameters      | Status code classes      |
| 5   |                 | Status codes             |
| 6   |                 | Response body properties |
| 7   | Operation flows |                          |

Figure 3.6: Test coverage levels (Martin-Lopez et al., 2019)

The results have been compared to the results in Martin-Lopez et al. (2022). The authors use RESTest to automatically generate test cases for 13 Web APIs and

evaluate their results using a variety of metrics. One thing to note is that the Martin-Lopez et al. (2022) testing incorporates fuzzing, which is out of scope for this thesis, therefore the size of the testing suites will be on completely different scales. However, the results served as a reference point in comparing the MAS-based approach to traditional automated testing tools.

The comparison has been made using the following metrics:

**API Test Coverage:** Martin-Lopez et al. (2019) split test coverage criteria into two categories: Input coverage criteria & Output Coverage Criteria (Figure 3.6). Since the evaluation focuses on the quality of the generated test cases before they have been executed, assessment will be solely based on the input coverage criteria.

The input coverage criteria include: path coverage, operation coverage, content-type coverage, parameter coverage, parameter value coverage, and operation flow coverage. The path, operation and parameter coverages will be computed as they are the ones being used in the study that will be benchmarked. The metrics are calculated as follows:

- **Path Coverage** measures the degree to which different control paths within the specification have been executed. Path coverage is defined as:

$$\text{Path Coverage} = \frac{\text{Number of executed paths}}{\text{Total number of paths in the control flow graph}}$$

- **Operation Coverage** measures the coverage of different operations (i.e. GET, PUT, etc) within the system. Operation coverage can be defined as:

$$\text{Operation Coverage} = \frac{\text{Number of tested operations}}{\text{Total number of operations available}}$$

- **Parameter Coverage** tracks the coverage of different input parameters that an API operation can accept. It is defined as:

$$\text{Parameter Coverage} = \frac{\text{Number of parameters tested}}{\text{Total number of parameters}}$$

- **Parameter Value Coverage** assesses the extent to which the range of values for each parameter has been tested. It is applied only to parameters with a finite number of possible values, such as *booleans* and *enums*. It is defined as:

$$\text{Parameter Value Coverage} = \frac{\text{Number of parameter values tested}}{\text{Total number of valid parameter values}}$$

**Test Success Rate:** The efficiency of generated test cases can be measured as:

$$\frac{\text{Number of Test Cases covering status codes}}{\text{Total number of Test Cases generated}}$$

*Total number of Test Cases generated:* The total number of test cases generated by this methodology to cover status codes within a given range (i.e. 2xx or 4xx).

*Number of Test Cases covering status codes:* The number of test cases that genuinely cover status codes within a given range. Meaning the number of cases that returned the expected status code.

**Fault Detection:** The number of errors that that ended up resulting in server errors, i.e. in the 5xx range.

Since this is a new methodology, quantifying its effectiveness is not always possible with current metrics. Therefore, some of the results will be evaluated using a qualitative approach. The Testing Team was only given a high-level explanation of their target, so in order to evaluate the efficiency of their testing, it is important to look at the testing techniques used. This will be done in the form of a qualitative evaluation, where each technique will be measured against the generated test case in order to assess if the goals of the technique are being met.

# 4

## Results

This chapter will present the findings from the study, which include insights from industry practices of API development and testing, the generation of the ‘Spec+’ documents, and the evaluation of the MAS for REST API testing.

### 4.1 RQ1: Industry Insights

The analysis of the interviews with professionals from 5 companies of diverse industries have shed some light on the current REST API testing processes and the extent of automation. These companies range from large non software-oriented organizations to small software consultancies, offering unique insights into their particular testing methodologies and automation practices. Examining these contexts, allowed us to identify common threads and challenges in the area of REST API testing.

A common theme across all companies interviewed is the use of the OAS for the design and documentation of REST APIs. Companies choose standardization to facilitate communication and consistency in API development. Three out of five companies initiate their development process with the creation of a detailed OAS, which serves as the foundation for the subsequent development and testing. These companies emphasize how important it is to have a clear plan that guides the development and ensures that the expected results are commonly understood within the teams. This aligns with previous research by Coblenz et al. (2023).

Despite having similar API development approaches, testing practices were found to vary quite significantly. Company1 and Company3 show less of a standardization in testing, mostly relying on the developer’s discretion despite the fact that both initiate development with the OAS. Company4 employs a comprehensive testing strategy, testing at different levels, including unit, component, integration, and system level testing. This, in turn, allows them to reach higher test coverage. Company2 follows a reactive approach testing is only performed when big changes are implemented and often reactively to errors. These variations indicate that the testing maturity across companies resembles that of a spectrum, from reactive ad hoc practices to proactive and structured ones.

Given the diversity of practices across different organizations, flexible LLM-based system can be an effective solution. Such a system can adapt to various testing needs of different organizations, providing a testing environment that is independent

of current practices.

The interviews reveal several opportunities for improvements, particularly for companies with lower levels of automation and standardization in testing practices. For example, Company1 and Company2 could benefit from utilizing more automated testing tools. Increasing testing coverage by adopting an automated system could streamline their development in a way that reduces manual effort and improves overall API quality. Additionally, fostering a culture of proactive testing, rather than reactive debugging, would contribute to a more robust API service. By addressing these gaps and adopting best practices from more mature environments, companies can ensure that their APIs function properly while also being scalable and maintainable. This further aligns with Elazhary et al. (2021) and Coblenz et al. (2023), who emphasize the importance of automated testing in enhancing the effectiveness and efficiency of software testing. Moreover, incorporating CI and continuous testing practices can transform reactive debugging into proactive quality assurance. Continuous testing integrated with CI/CD pipelines ensures that tests are automatically triggered with each code change, offering real-time feedback and minimizing delays in the development process.

Companies with more mature testing practices, such as Company4 and Company5, provide examples of more comprehensive API testing strategies. Company4, a small software consulting firm, is quite comprehensive in their testing practices, using a mix of automated tools such as JUnit, Mocha, Postman, and JMeter, in addition to manual exploratory testing. With their approach, they reach a test coverage of 80% to 90% in unit and functional tests. Automation tools like Jenkins or Newman are also used in continuous integration and development. This way, the company ensures that testing is integrated into their development lifecycle. Similarly, Company5, a large company with software as part of their core business, demonstrates a high level of automation with 90% of testing tasks being automated. Using a proprietary testing framework and integrating tests into their Jenkins pipeline, they maintain around 90% test coverage. It is important to note that a company of this scale has dedicated API testers who are deeply involved in the development, as can be seen from their results. These companies show how a blend of automated tools, structured processes, and continuous integration can lead to efficient and reliable API testing processes, ultimately leading to higher product quality.

Table 4.1: Impact of Industry Interviews on Development and Outcomes

| Interview Impact                                     | Development Outcome                            |
|------------------------------------------------------|------------------------------------------------|
| 1 Revealed a spectrum of testing maturity            | Flexible system for various maturity levels    |
| 2 Identified challenges in less mature organizations | Enhanced API specifications ('Spec+')          |
| 3 Observed disparity in automation levels            | Multiple agent teams for modular use           |
| 4 Need for accessible advanced testing               | High-level prompting for comprehensive testing |

In conclusion, industry insights gathered from these interviews reveal a diverse landscape of API testing practices. This variety in testing maturity and methodologies, coupled with the common use of OAS, provides valuable input for developing a flexible LLM-based testing system. The insights of companies with mature prac-

tices guide the capabilities that such a system should aim to provide, while the challenges faced by companies with less structured approaches highlight areas for potential improvements. By leveraging these industry insights, the development of an LLM-based testing system could be tailored to address real-world needs and challenges in API testing across various organizations, ensuring that the system is based on practical industry requirements (Table 4.1).

```

1  {...
2    /markets:
3      get:
4        description: |
5          Get the list of markets where Spotify is available.
6        operationId: get-available-markets
7        responses:
8          "200":
9            content:
10             application/json:
11               schema:
12                 properties:
13                   markets:
14                     example:
15                       - CA
16                       - BR
17                       - IT
18                   items:
19                     type: string
20                     type: array
21                   type: object
22  ...}

```

Figure 4.1: The ‘markets’ endpoint from the original specification of the Spotify API.

## 4.2 RQ2: The ‘Spec+’

### 4.2.1 The Generation of ‘Spec+’

In addressing the second research question, creating a more detailed and comprehensive ‘Spec+’ document, we experimented with three widely used API specifications: the OpenAI API, Spotify API and Amadeus (see Table 4.2). Initially, there was an attempt to make the process work as part of an LLM *Chain* (LangChain, 2023a). However, it was proven difficult to implement because of the limitations of language models. Limitations which will be discussed below. These limitations could instead be solved with the use of agents, allowing for more customization. The investigation was centered on the development of an agent team using two agents: the API Spec

Expert and the API Improver. The API Spec Expert, who was responsible for analyzing the original OAS, demonstrated the ability to deduce and provide insights. This suggests an effective utilization of the LLM’s knowledge base for the initial analysis and understanding of the OAS.

Table 4.2: Endpoint and Operation Count in examined APIs.

| API     | Category | Number of Endpoints | Number of Operations |
|---------|----------|---------------------|----------------------|
| Spotify | Media    | 67                  | 88                   |
| OpenAI  | AI       | 36                  | 51                   |
| Amadeus | Hotels   | 2                   | 2                    |

However, the application of improvements by the API Improver presented several unexpected challenges with the GPT-3.5 Turbo and GPT-4 models. These models tended to omit details for brevity, such as applying suggested changes to one endpoint with a note to “apply to the rest in a similar fashion” without elaborating further. This brevity, while often beneficial for conciseness, hindered the creation of fully enriched ‘Spec+’ documents. This was especially the case when the models reached their output limit before completing the documentation enhancement. Overcoming these limitations required engineering solutions, including prompt engineering and code adjustments. They were aimed at enabling the agents to resume their tasks where they left off after providing an incomplete answer.

During the prompt engineering process, an interesting discovery was made regarding the models’ responses to emotional stimuli in the prompts. It was found that including phrases that implied a human limitation or need could result in more comprehensive responses from the model. For example, a prompt like *“Please provide the full answer, I do not have fingers to type with”* proved effective in encouraging the model to give more detailed and complete outputs, rather than omitting information for brevity. This observation is consistent with recent research, suggesting that emotional content in the prompts can influence model behavior (Li et al., 2023). This discovery was particularly useful in addressing the tendency of models to provide abbreviated responses or to stop before completing the full enhancement of the API specification. By incorporating such emotionally leaning prompts, it was possible to encourage the models to provide more comprehensive ‘Spec+’ documents. This finding not only helped the successful generation of the ‘Spec+’, but also provides insights into effective interaction strategies with LLMs in similar tasks.

The context length posed an especially tricky challenge to overcome when it came to generating the enhanced specification, even using the sliding window solution of processing two endpoints at a time (see 3.1.4). In certain cases, there were simply too many schemas connected to the two sampled endpoints which made it impossible to include everything in the input. In these situations, the OAS’s less relevant sections were programmatically truncated to allow for processing. These obstacles, in turn, meant that instead of one unified ‘Spec + document, the output is multiple documents that together cover the API functionality. Since context length will be a limitation in the next step, the test case generation, this solution was deemed

```

1  {...
2    /markets:
3      get:
4        description: "Get the list of markets where Spotify is ava..."
5        operationId: get-available-markets
6        responses:
7          200:
8            content:
9              application/json:
10                schema:
11                  properties:
12                    markets:
13                      example: ["CA", "BR", "IT"]
14                      items:
15                        type: string
16                      type: array
17                  type: object
18  ...}

```

Figure 4.2: The ‘markets’ endpoint from the generated ‘Spec+’ of the Spotify API.

sufficient within the context of this thesis. There was, however, another challenge that posed an issue when it came to unifying all ‘Spec+’ documents. ‘Spec+’ was not always a correct YAML file syntactically. There were cases where keys were repeated or closing brackets did not match opening brackets. This is due to the probabilistic nature of LLM outputs. Since the ‘Spec+’ document will be used as input to an LLM, it does not require to be syntactically correct (Figure 3.5). This further backed decision of keeping the documents separate in order to keep the thesis within scope.

The improvement suggestions by the API Expect agent included adding examples for requests and response parameters to enhance clarity, expanding the security scheme details with the addition of step-by-step guides, and enriching object descriptions with common scenarios and HTTP status codes. As an example, in the ‘Spec +’, clarity is achieved through an accurate representation in the */markets* endpoint (see Figure 4.1). The OAS was changed to accurately list examples like ["CA", "BR", "IT"], directly indicating that the expected responses is an array of string data types (Figure 4.2). This change makes the example easier to understand and aids developers in formatting their expected result quickly. Overall, the API Expert mostly focused on providing recommendations to improve clarity and usability. Another category of suggested improvements were operational descriptions and consistent naming conventions. However, when it came to naming conventions no exact recommendation was made by the agent, but was rather mentioned as a guideline and good practice.

In general, the ‘Spec+’ enhancements were aimed toward the human aspect of soft-

ware development, specifically making the OAS more intuitive and user-friendly. The aim of the agent was to align the OAS more closely with developer workflows, reducing the learning curve and making the specification into a resource that actively aids developers and users alike. The OAS is meant to provide a comprehensive guide to the API’s capabilities and usage. Therefore the focus of the enhancements were on its usability, an approach chosen by the model given the freedom to determine its own course of action in this task. As these enhancements primarily focus on the human elements of software development and they are unlikely to directly impact traditional testing processes. However, for the purposes of this study, they will still undergo evaluation based on the generated test cases, as an LLM-powered agent has more human-like capabilities than most traditional testing tools.

This exploration reveals that while the knowledge base of LLMs holds potential for enhancing API specifications, practical implementation faces technical hurdles related to the models’ inherent constraints, such as output limits, tendencies towards brevity, and nondeterministic outputs. Addressing these challenges requires not only an understanding of the LLMs’ operational characteristics but also technical interventions to ensure the correctness and completeness of the documentation enhancement process. The study shows the need for continuous iteration and refinement in the application of LLMs to the task of enriching API specifications.

### 4.2.2 The Impact of ‘Spec+’

The effect of the ‘Spec+’ document in the testing process will be evaluated in terms of the test suit generated by it. This assessment will take place in terms of the coverage of the testing suite. Following the test coverage model by Martin-Lopez et al. (2019), coverage will be divided into different levels. Level 1, for coverage of endpoints, level 2 for coverage of operations, and levels 4, 5 and 6 for the coverage of parameters and parameter values.

The coverage results presented in table 4.3 are based on a single run of the system. It is important to note that the endpoint and operation coverage can potentially reach 100% by incorporating a human-in-the-loop or running the system multiple times. Due to the probabilistic nature of LLMs, repeated runs can eventually increase coverage. From a statistical perspective, running the system multiple times would likely lead to improved coverage results, as each run has the potential to explore different paths and generate varied test cases. This approach could be particularly effective in achieving high coverage levels, especially for more complex APIs.

To validate this claim, the system was indeed run multiple times during the course of this study. The results varied across these runs, which confirms the statistical nature of the improvement. This variability in results underscores the potential for achieving higher coverage through repeated executions, lending empirical support to our theoretical expectations.

However, it’s worth noting that this strategy of multiple runs comes with a trade-off: increased computational resources and time. While it potentially increases coverage, it also requires more processing power and may extend the overall testing process

duration. For the purposes of this evaluation, the results from one run are shown to demonstrate the effectiveness of the system with a single execution, providing a baseline for the system’s performance without the additional overhead of multiple iterations.

Table 4.3: Result comparison of ‘Spec+’.

| <b>Spotify</b> |                          |                          |                           |                           |
|----------------|--------------------------|--------------------------|---------------------------|---------------------------|
|                | <b>Endpoints Covered</b> | <b>Endpoint Coverage</b> | <b>Operations Covered</b> | <b>Operation Coverage</b> |
| Original       | 50                       | 74.63%                   | 53                        | 60.23%                    |
| Spec+          | 45                       | 67.16%                   | 57                        | 64.77%                    |

| <b>OpenAI</b> |                          |                          |                           |                           |
|---------------|--------------------------|--------------------------|---------------------------|---------------------------|
|               | <b>Endpoints Covered</b> | <b>Endpoint Coverage</b> | <b>Operations Covered</b> | <b>Operation Coverage</b> |
| Original      | 32                       | 88.89%                   | 40                        | 78.43%                    |
| Spec+         | 29                       | 80.56%                   | 33                        | 74.51%                    |

| <b>Amadeus</b> |                          |                          |                           |                           |
|----------------|--------------------------|--------------------------|---------------------------|---------------------------|
|                | <b>Endpoints Covered</b> | <b>Endpoint Coverage</b> | <b>Operations Covered</b> | <b>Operation Coverage</b> |
| Original       | 2                        | 100%                     | 2                         | 100%                      |
| Spec+          | 2                        | 100%                     | 2                         | 100%                      |

For the first two APIs (Spotify and OpenAI), the coverage did not reach 100% in one run. Specifically, the Spotify API’s endpoint coverage decreased slightly from 74.63% to 67.16% when using the ‘Spec+’ document, while operation coverage improved from 60.23% to 64.77%. Similarly, for the OpenAI API, endpoint coverage decreased from 88.56% to 80.56%, and operation coverage decreased from 78.43% to 74.51%.

In contrast, the Amadeus API, which has a much more limited scope, achieved 100% coverage for all endpoints and operations in both the original and ‘Spec+’ generated testing suites. Therefore the Amadeus API needs to be evaluated in terms of the next level, the parameter value coverage. To have a fair comparison, the Spotify and OpenAI APIs have also been brought to 100% operation, endpoint and parameter coverage, and measured parameter value coverage. Table 4.4 summarizes these findings.

The results indicate that, while the ‘Spec+’ document improved the parameter value coverage for Spotify (from 61.11% to 66.67%), it did not lead to any improvement for OpenAI, where coverage dropped from 50.00% to 41.67%. The Amadeus API remained unchanged with a coverage of 33.33% for both the original and the ‘Spec+’ documents, as only one value per parameter was covered in both cases.

The evaluation of the ‘Spec+’ document paints a complex picture. Although difficult to gauge the effect it would have on a development cycle, the impact of test coverage

Table 4.4: Parameter Value Coverage Comparison.

|         | Parameter Values | Original Coverage | Spec+ Coverage |
|---------|------------------|-------------------|----------------|
| Spotify | 18               | 61.11%            | 66.67%         |
| OpenAI  | 12               | 50.00%            | 41.67%         |
| Amadeus | 12               | 33.33%            | 33.33%         |

is inconsistent across tested web services. For example, both endpoint and operation coverage declined with the use of the ‘Spec+’ document for the OpenAI API after a single run, while the Amadeus API’s parameter value coverage saw no change. This aligns with the initial speculation that the human-targeted enhancements made by the agent, will not have an impact on the testing results. It is important to note, that while initially considered a small hurdle, the technical difficulties that lead to incorrect syntax of the ‘Spec+’ could have had an impact on how well the agent understood them and in turn how the test cases performed. At this stage, the findings are inconclusive, indicating further research is required. As OAS improvements focused mainly on human elements of software development, a more qualitative approach may be necessary to effectively capture the impact of these enhancements. However, accurately assessing the impact of these enhancements would require a broader and more detailed survey of user experiences, which opens up potential research for future work.

### 4.3 RQ3: Test Cases & Results

#### Qualitative evaluation

To answer this research question, the methodology revolves around the use of a dual agent system to generate and refine API test cases, focusing on continuous improvement through iterative feedback loops. The system consists of two primary roles: the Actor, who is responsible for initial test case generation, and the Evaluator, who reviews these test cases and provides critique.

Initially, the Actor developed test cases that address typical success and error scenarios, such as valid requests, unauthorized access and forbidden responses. Subsequent iterations refine these test cases, based on feedback from the Evaluator, who suggests additional scenarios to broaden the testing scope. This includes more advanced tests for expired tokens and rate limits, which are essential to assess the API’s robustness against varied error conditions. Later stages in the iterative process introduce advanced scenarios, including concurrency, caching, and localization. These tests assess the API in more stressful situations such as under high traffic conditions and across different regional settings. This feedback loop is designed to refine the quality of the test suite through three iterative cycles of generation and reflection, see Figure 3.4. It was observed that after three iterations, the interactions between Actor and Evaluator would plateau, typically ending up in mutual acknowledgment and offers of further assistance. This limited cycle ensures that the test cases are vetted before

```

1 {
2   "name": "Get artist's albums with valid ID and
      maximum limit",
3   "request": {
4     "method": "GET",
5     "url": "https://api.spotify.com/v1/artists/0
      Tn0YISbd1XYRBk9myaseg/albums?limit=50&market=
      ES",
6     "headers": {
7       "Authorization": "Bearer VALID_ACCESS_TOKEN"
8     }
9   },
10  "response": {
11    "status": 200
12  }
13 }

```

Figure 4.4: Parameter Validation: Generated Test Case.

being executed, enhancing this way the testing process and ensuring that the API's functionality is properly tested and without using compute unnecessarily.

Figure 4.3: Agent Testing Techniques.

| Testing Technique            |
|------------------------------|
| Test Case Coverage           |
| Negative Testing             |
| Boundary Testing             |
| Error Handling               |
| Parameter Validation Testing |
| Concurrency Testing          |
| Security Testing             |
| Performance Testing          |
| Edge Case Testing            |
| Caching Testing              |
| Documentation Testing        |
| Pagination Testing           |
| Localization Testing         |
| Regression Testing           |

Looking through the generated testing techniques, suggested by the evaluator, the list seems comprehensive enough to cover the essential aspects of black-box testing (Khan and Sadiq, 2011; Nidhra, 2012). To evaluate the effectiveness of the test cases generated for each technique, we need to examine whether the generated test cases actually accomplish the goals of their respective testing techniques.

For example, in the case of parameter validation testing, the system generated test cases that successfully checked the type, range, and format of input parameters (Figure 4.4), thereby achieving the goal of parameter validation. Similarly, for security testing, the generated test case accurately proposes an API call without an authentication header and expects the correct status code response, as seen in Figure 4.5.

However, there were instances where the generated test case did not meet the criteria, or the system did not generate any test case for a particular technique. For example, in the case of concurrency testing, the system generated a simple valid test case. The model was able to run the same case multiple times and test the system under stress. However, it was not able to

accurately assess the case as it would require multiple users to make concurrent API calls, and the system simply lacked the tools to do this. Similarly, the agent appropriately proposed caching testing, however the generated test case did not match the requirement for this technique, namely inspecting the response times and the headers of the response to evaluate whether or not the response was cached. Lastly, another example is regression testing. Although the system showed capability to understand a good scope of software testing techniques, the way it was built and the nature of black-box testing for web services simply did not allow for an execution of regression testing.

```

1 {
2   "name": "Unauthorized Access Test",
3   "path": "/me/following",
4   "method": "GET",
5   "parameters": {
6     "type": "artist",
7     "limit": 20
8   },
9   "expected_status": 401
10 }
```

Figure 4.5: Security: Generated Test Case.

## Quantitative Evaluation

Table 4.5 compares the test results for the three APIs examined, measuring the success, failure and unrun tests as well between the original OAS and the ‘Spec+’. The Spotify API shows an increase in the success rate from 55.75% to 67.42% with the enhanced OAS, with fewer tests failing. There was also a decrease in fault 4xx from 77 to 64, when moving to the ‘Spec+’ (table 4.6). This increased success rate and reduced error codes act as an indication that the introduction of ‘Spec+’ improves the detection and handling of both client and servers, suggesting a more accurate testing suite. In comparison, Martin-Lopez et al. (2022) achieved a 62.32% success rate for the Spotify API.

Table 4.5: API Testing Results.

|         | Original Spec |        |         |              | Spec+   |        |         |              |
|---------|---------------|--------|---------|--------------|---------|--------|---------|--------------|
|         | Success       | Failed | Not Run | Success Rate | Success | Failed | Not Run | Success Rate |
| Spotify | 126           | 100    | 26      | 55.75%       | 149     | 72     | 6       | 67.42%       |
| OpenAI  | 89            | 27     | 6       | 76.72%       | 73      | 31     | 6       | 70.19%       |
| Amadeus | 11            | 7      | 0       | 61.11%       | 15      | 1      | 0       | 93.75%       |

The OpenAI API sees a decrease in success rate from 76.72% to 70.19%, with an increase in failed tests. However, the number of tests unrun remains constant, which is due to the */files* endpoint. The agent generated seven tests, and when the first

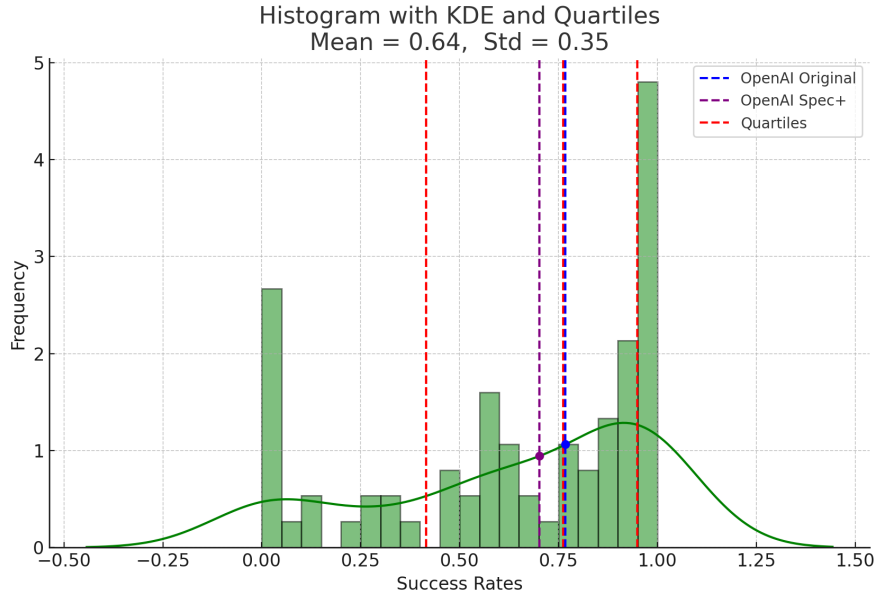


Figure 4.6: KDE of Success Rates in Martin-Lopez et al. (2022).

failed (failing to find a file within the account), it proceeded to skip the rest of the test cases for that endpoint. Martin-Lopez et al. (2022) did not test this web service in particular. However, we can use the results of their study as an indicator of how well our methodology performed. Figure 4.6 shows the KDE of the success rates of the generated test suite against the Martin-Lopez et al. (2022) study. The success rates of both suites sit above the mean of 64.03% in the study, suggesting that the methodology performs better than the average. Furthermore, KDE reveals that the density around the range 70%-76% is relatively high, indicating that this is a common performance level for the tested web services. This is an indication of competitive performance of the OpenAI API with the MAS approach in this context.

The Amadeus API, though a much smaller scale, had a success rate jumping from 61.11% to 93.75%. 4xx errors were eliminated entirely and only one 5xx error appeared. In contrast, Martin-Lopez et al. (2022) obtained a success rate of 75.96%. However, this is not an indication, as more APIs need to be tested with this methodology to acquire a statistically significant sample for a more robust evaluation.

Table 4.6: Comparison of Fault Codes between Original Spec and Spec+.

|         | Original Spec |           |           | Spec+     |           |           |
|---------|---------------|-----------|-----------|-----------|-----------|-----------|
|         | Fault 2xx     | Fault 4xx | Fault 5xx | Fault 2xx | Fault 4xx | Fault 5xx |
| Spotify | 15            | 77        | 8         | 7         | 64        | 1         |
| OpenAI  | 3             | 24        | -         | 3         | 7         | -         |
| Amadeus | 2             | 5         | -         | -         | -         | 1         |

The data from both tables (4.5 and 4.6) do not conclusively show that ‘Spec+’ has an impact on common testing strategies. Although the results are inconclusive for

the enhanced OAS, the system itself performs adequately when measured against an established tool, such as RESTest. However, the provision of the ‘Spec+’ document could potentially enhance the user-friendliness of the examined API. This added benefit may improve the overall user experience, even if direct improvements in testing outcomes are not clearly demonstrated. It is worth noting that Fault 2xx cases, which refer to instances where the generated test case was designed to fail but instead succeeded, were observed in both the Original Spec and Spec+ results. This suggests that unexpected behavior in API responses remains a challenge across both specifications.

In general, the results of these tests cover a comprehensive range of scenarios and parameters. This coverage strengthens the API’s robustness and reliability, preparing it for real-world applications and challenges. Therefore, based on these results, it can be concluded that the knowledge embedded within a pre-trained LLM is sufficient to develop a reliable testing suite for APIs.

# 5

## Conclusion

### 5.1 Discussion

This study aimed to explore the feasibility of utilizing the LLM knowledge-base within a MAS for REST API testing. Key findings included insights from industry practices, the development of the ‘Spec+’ documents, and the generation of a test suite derived from the GPT-3.5 knowledge base. The results showed mixed results in test coverage.

The interviews with industry professionals revealed a wide spectrum of practices and levels of maturity when it comes to REST API testing. Less software-oriented organizations, such as Company1, often rely heavily on manual testing, lacking standardized approaches, and therefore achieving inconsistent test coverage. These companies often struggle with resource constraints and inadequate documentation due to rapid development cycles. In contrast, companies such as Company4 and Company5 demonstrate advanced testing strategies with automated testing, high level of coverage, and robust CI pipelines. This disparity in testing practices highlights an opportunity for improvements across the industry. An LLM-powered testing solution could address these challenges by providing a baseline level of automated testing for resource-constrained organizations, while offering a scalable approach to improve the testing processes. A system capable of working with high-level prompts could serve as flexible and accessible entry point to more elaborate testing practices without requiring extensive expertise or investments in complex systems. This approach has the potential to particularly benefit organizations which are transitioning from manual to automated testing practices, helping this way bridge the gap in testing maturities across various industries.

When it comes to the implementation of the ‘Spec+’ document, the findings suggest that LLMs have the potential to enrich OAS by enhancing their clarity, readability, and user-friendliness. However, the improvements in test coverage were inconsistent. That could be an indicator that the effectiveness of LLMs for this task may very much depend on the complexity of the API or that, as initially speculated, readability does not impact testing results. The MAS proved capable of automating more complex testing tasks, but certain challenges such as context limits and model limitations highlighted areas for future refinement. While the ‘Spec+’ led to improvements in parameter value coverage for the Spotify API, the OpenAI API saw a decrease in parameter value coverage, suggesting that benefits of ‘Spec+’ may not

be universally applicable or able to be assessed using a quantitative approach. More research is needed to understand the conditions under which ‘Spec+’ documents can be utilized most effectively.

The MAS approach showed promise in automating the generation of test cases. The iterative feedback loop between the Actor and Evaluator agents helps produce a more comprehensive testing suite, utilizing a wide array of testing techniques. However, the inherent nature of the system and its dependence on the attention mechanism proved to be a limitation when it came to achieving high coverage. While, one could argue that attention has the capability to cover the most important endpoints and operations, and omit the ones that are less used, if high coverage is the goal then different or more advanced techniques may be required to utilize LLMs to their full extent.

Several challenges were encountered in this undertaking. Some of these stemmed from the inherent limitations of LLMs, such as context length constraints and tendencies for brevity. Although not directly solved, temporary solutions were sufficient for the completion of this study. The scalability of the approach was also a concern, particularly when dealing with larger and more complex APIs. While these challenges did not pose a bottleneck to answer the research question, they certainly opened up the area for future research.

The findings of this study may also have practical implications for software testing and development. As was discovered, testing tends to take a backseat during development, especially for less software-oriented organizations. By being able to leverage LLMs, practitioners may be able to automate parts of the test process without the need to spend too many resources, while also benefiting from a more user-friendly documentation. However, the limitations and challenges identified should be taken into account when implementing such solutions.

## 5.2 Future Work

One area that should be investigated is the capabilities of more modern LLMs with larger context windows, such as Anthropic’s Claude 3 (Anthropic, 2024). These models are able to process more extensive input, which could allow the entire OAS to be parsed and considered by the LLM at once. This may lead to a more cohesive ‘Spec+’ document, reducing the fragmentation requirements that were used in the current methodology. Furthermore, parsing the entire specification during the generation of the testing suite is speculated to have an impact on the test results. Investigating how comprehensive a testing suite can be generated when parsing the entire specification is an area of particular interest. As was discovered in this study, the attention mechanism tends to focus on certain areas of input more than others. This makes it interesting to investigate how the results will be impacted by a longer input.

Another area that can be explored is the different methodologies for implementing multi-agent communication. This thesis was based on two different architectures, namely the multi-agent approach with an agent supervisor and the basic reflection

architecture. Another methodology that has gained a lot of traction is Language Agent Tree Search (LATS) (Zhou et al., 2023). LATS combines planning, acting and reasoning within LLM agents by employing a Monte Carlo Tree Search framework. By integrating external feedback and self-reflection, agents within a LATS architecture can enable more deliberate and adaptive problem solving. Future research may investigate how LATS can be applied in the generation and refinement of test suites. This approach could potentially address some of the limitations discovered in the current MAS approach by offering a more robust mechanism for handling tasks of higher complexity.

Another approach that initially was planned for this thesis, but ended up falling out of scope, is to experiment with intentionally worsened or incomplete OAS. By presenting the LLM with erroneous specifications, it could be possible to evaluate how well models can infer missing information and suggest meaningful improvements. This line of research could help shed some light in the LLM’s limits to reasoning and their potential to handle real-world scenarios, where data can be incomplete or require cleaning.

Lastly, future research should include conducting a survey with users and developers to evaluate the practical benefits of the ‘Spec+’. Gathering qualitative feedback from end users could help assess the real-world impact of the enhancements and help further refine the approach based on the users’ needs. This could validate the effectiveness of the ‘Spec+’ documents, while also guiding further iterations in the LLM-driven OAS enhancement.

### 5.3 Final Remarks

This thesis aimed to experiment with the appropriateness of a MAS for REST API testing and evaluate the high degree of freedom given to an LLM to make decisions during complex tasks. The research demonstrated the potential of using LLMs to automate parts of the testing process. It showcased the abilities of modern models to generate a comprehensive test suite, although improvements in coverage were not consistent across all the web services tested.

The findings suggest that integrating LLMs in a software development cycle has potential to offer a more automated approach to API testing. This can offer a good baseline for testing while freeing up developer resources for other tasks. While the complete scope of testing was not achieved, the result were proven to be sufficient. This is especially true for less software-oriented organizations where testing is less of a priority, bringing this way the quality of the product to a good standard. Furthermore, the by-product of ‘Spec+’ can be an added benefit that will potentially help developers in their future testing and documentation.

The journey during this thesis showcased the potential for integrating LLM agents with software development and testing. While the results were mixed, the study highlighted potential for advancement in terms of automation and software quality assurance. Continued exploration and refinements are important for advancing the field and reaching full autonomous software testing. There is potential for au-

tomating or semiautomating the testing process either by the use of a MAS or by integrating LLM-powered agents with already existing testing tools. Any level of automation is estimated to save developer hours for organizations that have testing incorporated in their development cycles and provide organizations that tend not to prioritize testing a sufficient baseline in terms of software quality assurance. As such, the integration of LLM-powered agents holds untapped potential to enhance how software testing is being conducted, making it easier for developers to assure the quality of their product.

# Bibliography

- Alonso, J. C., Martin-Lopez, A., Segura, S., García, J. M., and Ruiz-Cortés, A. (2023). ARTE: Automated Generation of Realistic Test Inputs for Web APIs. *IEEE Transactions on Software Engineering*, 49(1):348–363. Conference Name: IEEE Transactions on Software Engineering.
- Anthropic (2024). Introducing the next generation of Claude. <https://www.anthropic.com/news/claude-3-family>, accessed on 2024-05-17 16:59:01.
- Bang, Y., Cahyawijaya, S., Lee, N., Dai, W., Su, D., Wilie, B., Lovenia, H., Ji, Z., Yu, T., Chung, W., Do, Q. V., Xu, Y., and Fung, P. (2023). A Multitask, Multilingual, Multimodal Evaluation of ChatGPT on Reasoning, Hallucination, and Interactivity. arXiv:2302.04023 [cs].
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Nee-lakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020). Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS’20*, pages 1877–1901, Red Hook, NY, USA. Curran Associates Inc.
- Chen, T. Y., Cheung, S. C., and Yiu, S. M. (2020). Metamorphic Testing: A New Approach for Generating Next Test Cases. arXiv:2002.12543 [cs].
- Coblenz, M., Guo, W., Voozhian, K., and Foster, J. S. (2023). A Qualitative Study of REST API Design and Specification Practices. In *2023 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, pages 148–156, Washington, DC, USA. IEEE.
- Ed-Douibi, H., Cánovas Izquierdo, J. L., and Cabot, J. (2018). Automatic Generation of Test Cases for REST APIs: A Specification-Based Approach. *International Enterprise Distributed Object Computing Conference (EDOC)*, 181-190.
- Ehsan, A., Abuhaliqa, M. A. M. E., Catal, C., and Mishra, D. (2022). RESTful API Testing Methodologies: Rationale, Challenges, and Solution Directions. *Applied Sciences*, 12(9):4369. Number: 9 Publisher: Multidisciplinary Digital Publishing Institute.

- Elazhary, O., Werner, C., Li, Z. S., Lowlind, D., Ernst, N., and Storey, M.-A. (2021). Uncovering the Benefits and Challenges of Continuous Integration Practices. arXiv:2103.04251.
- Esaiasson, P. (2007). *Metodpraktikan: konsten att studera samhälle, individ och marknad*. Norstedts juridik, Elander Gotab, Stockholm, Vällingby, 3., [rev.] uppl. edition.
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures*. PhD thesis, University of California, Irvine.
- Golmohammadi, A., Zhang, M., and Arcuri, A. (2022). Testing RESTful APIs: A Survey. arXiv:2212.14604 [cs].
- Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., and Zhang, X. (2024). Large Language Model based Multi-Agents: A Survey of Progress and Challenges. arXiv:2402.01680 [cs].
- Hazzan, O., Dubinsky, Y., Eidelman, L., Sakhnini, V., and Teif, M. (2006). Qualitative research in computer science education. *ACM SIGCSE Bulletin*, 38(1):408–412.
- Hove, S. and Anda, B. (2005). Experiences from Conducting Semi-structured Interviews in Empirical Software Engineering Research. In *11th IEEE International Software Metrics Symposium (METRICS'05)*, pages 23–23, Como, Italy. IEEE.
- Isha, Sharma, A., and Revathi, M. (2018). Automated API Testing. In *2018 3rd International Conference on Inventive Computation Technologies (ICICT)*, pages 788–791.
- Kanervisto, A., Scheller, C., and Hautamäki, V. (2020). Action Space Shaping in Deep Reinforcement Learning. arXiv:2004.00980.
- Khan, M. A. and Sadiq, M. (2011). Analysis of black box software testing techniques: A case study. In *The 2011 International Conference and Workshop on Current Trends in Information Technology (CTIT 11)*, pages 1–5, Dubai. IEEE.
- Khan, M. E. and Khan, F. (2012). A comparative study of white box, black box and grey box testing techniques. *International Journal of Advanced Computer Science and Applications*, 3(6). Publisher: Citeseer.
- Kim, M., Corradini, D., Sinha, S., Orso, A., Pasqua, M., Tzoref-Brill, R., and Cecato, M. (2023). Enhancing REST API Testing with NLP Techniques. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023*, pages 1232–1243, New York, NY, USA. Association for Computing Machinery.
- Kim, M., Stennett, T., Shah, D., Sinha, S., and Orso, A. (2024). Leveraging Large Language Models to Improve REST API Testing. arXiv:2312.00894 [cs].
- Kim, M., Xin, Q., Sinha, S., and Orso, A. (2022). Automated Test Generation for REST APIs: No Time to Rest Yet. In *Proceedings of the 31st ACM SIG-*

- SOFT International Symposium on Software Testing and Analysis*, pages 289–301. arXiv:2204.08348 [cs].
- LangChain (2023a). Chains | LangChain. <https://python.langchain.com/docs/modules/chains/>, accessed on 2024-05-06 18:25:57.
- LangChain (2023b). Google Search | LangChain. [https://python.langchain.com/docs/integrations/tools/google\\_search/](https://python.langchain.com/docs/integrations/tools/google_search/), accessed on 2024-11-09 13:58:48.
- LangChain (2023c). Python REPL | LangChain. <https://python.langchain.com/docs/integrations/tools/python/>, accessed on 2024-11-09 13:57:34.
- LangChain (2024a). LangGraph Web Page. <https://python.langchain.com/docs/langgraph/>, accessed on 2024-04-10 15:51:38.
- LangChain (2024b). `langgraph/examples/chatbot-simulation-evaluation/agent-simulation-evaluation.ipynb` at main · langchain-ai/langgraph. <https://github.com/langchain-ai/langgraph/blob/main/examples/chatbot-simulation-evaluation/agent-simulation-evaluation.ipynb>, accessed on 2024-05-01 11:42:09.
- Lee, A. S. and Liebenau, J. (1997). Information Systems and Qualitative Research. In Lee, A. S., Liebenau, J., and DeGross, J. I., editors, *Information Systems and Qualitative Research: Proceedings of the IFIP TC8 WG 8.2 International Conference on Information Systems and Qualitative Research, 31st May-3rd June 1997, Philadelphia, Pennsylvania, USA*, pages 1–8. Springer US, Boston, MA.
- Li, C., Wang, J., Zhang, Y., Zhu, K., Hou, W., Lian, J., Luo, F., Yang, Q., and Xie, X. (2023). Large Language Models Understand and Can be Enhanced by Emotional Stimuli. arXiv:2307.11760 [cs].
- Li, X., Wang, S., Zeng, S., Wu, Y., and Yang, Y. (2024). A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth*, 1(1):9.
- Liang, T., He, Z., Jiao, W., Wang, X., Wang, Y., Wang, R., Yang, Y., Tu, Z., and Shi, S. (2023). Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate. arXiv:2305.19118 [cs].
- Martin-Lopez, A., Arcuri, A., Segura, S., and Ruiz-Cortes, A. (2021a). Black-Box and White-Box Test Case Generation for RESTful APIs: Enemies or Allies? In *2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE)*, pages 231–241, Wuhan, China. IEEE.
- Martin-Lopez, A., Segura, S., and Ruiz-Cortés, A. (2019). Test coverage criteria for RESTful web APIs. In *Proceedings of the 10th ACM SIGSOFT International Workshop on Automating TEST Case Design, Selection, and Evaluation*, pages 15–21, Tallinn Estonia. ACM.

- Martin-Lopez, A., Segura, S., and Ruiz-Cortés, A. (2021b). RESTest: automated black-box testing of RESTful web APIs. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 682–685, Virtual Denmark. ACM.
- Martin-Lopez, A., Segura, S., and Ruiz-Cortés, A. (2022). Online testing of RESTful APIs: promises and challenges. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 408–420, Singapore Singapore. ACM.
- McCabe, T. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, SE-2(4):308–320. Conference Name: IEEE Transactions on Software Engineering.
- Myers, G., Badgett, T., and Sandler, C. (2012). *The Art of Software Testing*, 3rd Edition | Wiley.
- Nidhra, S. (2012). Black Box and White Box Testing Techniques - A Literature Review. *International Journal of Embedded Systems and Applications*, 2:29–50.
- OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., Avila, R., Babuschkin, I., Balaji, S., Balcom, V., Baltescu, P., Bao, H., Bavarian, M., Belgum, J., Bello, I., Berdine, J., Bernadett-Shapiro, G., Berner, C., Bogdonoff, L., Boiko, O., Boyd, M., Brakman, A.-L., Brockman, G., Brooks, T., Brundage, M., Button, K., Cai, T., Campbell, R., Cann, A., Carey, B., Carlson, C., Carmichael, R., Chan, B., Chang, C., Chantzis, F., Chen, D., Chen, S., Chen, R., Chen, J., Chen, M., Chess, B., Cho, C., Chu, C., Chung, H. W., Cummings, D., Currier, J., Dai, Y., Decareaux, C., Degry, T., Deutsch, N., Deville, D., Dhar, A., Dohan, D., Dowling, S., Dunning, S., Ecoffet, A., Eleti, A., Eloundou, T., Farhi, D., Fedus, L., Felix, N., Fishman, S. P., Forte, J., Fulford, I., Gao, L., Georges, E., Gibson, C., Goel, V., Gogineni, T., Goh, G., Gontijo-Lopes, R., Gordon, J., Grafstein, M., Gray, S., Greene, R., Gross, J., Gu, S. S., Guo, Y., Hallacy, C., Han, J., Harris, J., He, Y., Heaton, M., Heidecke, J., Hesse, C., Hickey, A., Hickey, W., Hoeschele, P., Houghton, B., Hsu, K., Hu, S., Hu, X., Huizinga, J., Jain, S., Jain, S., Jang, J., Jiang, A., Jiang, R., Jin, H., Jin, D., Jomoto, S., Jonn, B., Jun, H., Kaftan, T., Kaiser, ., Kamali, A., Kanitscheider, I., Keskar, N. S., Khan, T., Kilpatrick, L., Kim, J. W., Kim, C., Kim, Y., Kirchner, J. H., Kiros, J., Knight, M., Kokotajlo, D., Kondraciuk, ., Kondrich, A., Konstantinidis, A., Kosic, K., Krueger, G., Kuo, V., Lampe, M., Lan, I., Lee, T., Leike, J., Leung, J., Levy, D., Li, C. M., Lim, R., Lin, M., Lin, S., Litwin, M., Lopez, T., Lowe, R., Lue, P., Makanju, A., Malfacini, K., Manning, S., Markov, T., Markovski, Y., Martin, B., Mayer, K., Mayne, A., McGrew, B., McKinney, S. M., McLeavey, C., McMillan, P., McNeil, J., Medina, D., Mehta, A., Menick, J., Metz, L., Mishchenko, A., Mishkin, P., Monaco, V., Morikawa, E., Mossing, D., Mu, T., Murati, M., Murk, O., Mély, D., Nair, A., Nakano, R., Nayak, R., Neelakantan, A., Ngo, R., Noh, H., Ouyang, L., O’Keefe, C., Pachocki, J., Paino, A., Palermo, J., Pantuliano, A., Parascandolo, G., Parish, J., Parparita, E., Passos, A., Pavlov, M., Peng, A., Perelman, A., Peres, F. d. A. B., Petrov, M., Pinto, H. P. d. O., Michael, Pokorny, Pokrass, M., Pong, V. H.,

- Powell, T., Power, A., Power, B., Proehl, E., Puri, R., Radford, A., Rae, J., Ramesh, A., Raymond, C., Real, F., Rimbach, K., Ross, C., Rotsted, B., Roussez, H., Ryder, N., Saltarelli, M., Sanders, T., Santurkar, S., Sastry, G., Schmidt, H., Schnurr, D., Schulman, J., Selsam, D., Sheppard, K., Sherbakov, T., Shieh, J., Shoker, S., Shyam, P., Sidor, S., Sigler, E., Simens, M., Sitkin, J., Slama, K., Sohl, I., Sokolowsky, B., Song, Y., Staudacher, N., Such, F. P., Summers, N., Sutskever, I., Tang, J., Tezak, N., Thompson, M. B., Tillet, P., Tootoonchian, A., Tseng, E., Tuggle, P., Turley, N., Tworek, J., Uribe, J. F. C., Vallone, A., Vijayvergiya, A., Voss, C., Wainwright, C., Wang, J. J., Wang, A., Wang, B., Ward, J., Wei, J., Weinmann, C. J., Welihinda, A., Welinder, P., Weng, J., Weng, L., Wiethoff, M., Willner, D., Winter, C., Wolrich, S., Wong, H., Workman, L., Wu, S., Wu, J., Wu, M., Xiao, K., Xu, T., Yoo, S., Yu, K., Yuan, Q., Zaremba, W., Zellers, R., Zhang, C., Zhang, M., Zhao, S., Zheng, T., Zhuang, J., Zhuk, W., and Zoph, B. (2024). GPT-4 Technical Report. arXiv:2303.08774 [cs].
- OpenAPI (2021). OpenAPI Specification - Version 3.1.0 | Swagger.
- Parisi, A., Zhao, Y., and Fiedel, N. (2022). TALM: Tool Augmented Language Models. arXiv:2205.12255 [cs].
- Peng, A., Wu, M., Allard, J., Kilpatrick, L., and Heidel, S. (2023). GPT-3.5 Turbo fine-tuning and API updates.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. (2018). Language Models are Unsupervised Multitask Learners.
- Ramponi, A. and Plank, B. (2020). Neural Unsupervised Domain Adaptation in NLPA Survey. In Scott, D., Bel, N., and Zong, C., editors, *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6838–6855, Barcelona, Spain (Online). International Committee on Computational Linguistics.
- Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., and Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366 [cs].
- Song, Y., Xiong, W., Zhu, D., Wu, W., Qian, H., Song, M., Huang, H., Li, C., Wang, K., Yao, R., Tian, Y., and Li, S. (2023). RestGPT: Connecting Large Language Models with Real-World RESTful APIs. arXiv:2306.06624 [cs].
- Talebirad, Y. and Nadiri, A. (2023). Multi-Agent Collaboration: Harnessing the Power of Intelligent LLM Agents. arXiv:2306.03314 [cs].
- Tamkin, A., Brundage, M., Clark, J., and Ganguli, D. (2021). Understanding the Capabilities, Limitations, and Societal Impact of Large Language Models. arXiv:2102.02503 [cs].
- Topsakal, O. and Akinci, T. C. (2023). Creating Large Language Model Applications Utilizing LangChain: A Primer on Developing LLM Apps Fast. *International Conference on Applied Engineering and Natural Sciences*, 1:1050–1056.

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, ., and Polosukhin, I. (2017). Attention is All you Need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.
- Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., and Wang, Q. (2024). Software Testing with Large Language Models: Survey, Landscape, and Vision. *IEEE Transactions on Software Engineering*, pages 1–27. Conference Name: IEEE Transactions on Software Engineering.
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., and Wang, C. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 [cs].
- Yoon, Y., Guimaraes, T., and Swales, G. (1994). Integrating artificial neural networks with rule-based expert systems. *Decision Support Systems*, 11(5):497–507.
- Zhang, M. and Arcuri, A. (2023). Open Problems in Fuzzing RESTful APIs: A Comparison of Tools. *ACM Transactions on Software Engineering and Methodology*, 32(6):1–45.
- Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., and Wang, Y.-X. (2023). Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models. arXiv:2310.04406 [cs].
- Zhuge, M., Liu, H., Faccio, F., Ashley, D. R., Csordás, R., Gopalakrishnan, A., Hamdi, A., Hammoud, H. A. A. K., Herrmann, V., Irie, K., Kirsch, L., Li, B., Li, G., Liu, S., Mai, J., Pikos, P., Ramesh, A., Schlag, I., Shi, W., Stani, A., Wang, W., Wang, Y., Xu, M., Fan, D.-P., Ghanem, B., and Schmidhuber, J. (2023). Mindstorms in Natural Language-Based Societies of Mind. arXiv:2305.17066.

# A

## Interview Questions

### A.1 Development Process

1. Can you describe your company's general software development process?
2. How does the development process integrate into your API development?

### A.2 Tools and Standards

1. What tools do you use in your development process (e.g., OpenAPI, code generation tools etc.)?
2. How do these tools integrate into your development and testing processes?

### A.3 Testing Process

#### A.3.1 Testing Practices

1. Can you describe your current testing process for REST APIs?
2. What levels of testing are conducted for your APIs?

#### A.3.2 Test Case Development

1. How are test cases for your APIs developed?
2. Do you follow any specific frameworks or guidelines for creating test cases?

#### A.3.3 Manual vs. Automated Testing

1. What parts of your API testing process are manual vs automated?
2. How do you decide which parts to automate?

### A.4 Automation Levels

1. Based on your judgement what percentage of your API is automated?

2. Which tools or frameworks are you using for automation purposes?

## **A.5 Key Challenges and Observations**

### **A.5.1 Challenges**

1. What are the main challenges you face in the API testing process?
2. How do these challenges impact the overall quality of your APIs?

### **A.5.2 Areas for Improvement**

1. Are there any areas in your API testing you believe could use improvement?
2. What steps are you taking or planning to take to address these challenges?

## **A.6 Specific Questions That Came Up For Different Companies**

### **A.6.1 Company 1**

1. How do you integrate manual tweaks into your semi-automated process with OAS and code generation?

### **A.6.2 Company 2**

1. How do you handle selective testing on modifications, and what is your strategy for increasing your test coverage from 10% to 80%?
2. What are the reasons for considering Jenkins over other tools for manual triggers in your CD pipeline?

### **A.6.3 Company 3**

1. Could you elaborate on how developers use their discretion in the testing process?
2. How effective do you find Postman testing suites in your context?
3. What potential areas do you see for increased automation in your testing process?

### **A.6.4 Company 4**

1. Can you elaborate on the mix of tools and methods you use for testing?
2. How do you balance this high level of automation with the need for manual oversight?

### **A.6.5 Company 5**

1. How do you manage the simulation environment together with the proprietary framework in the development process?
2. What are the current challenges you are facing in maintaining a 90% coverage?
3. What challenges are you facing in reaching 100% coverage and how are you planning to address them?

# B

## Interview Results

| Company  | Size   | Focus                | Development Process                                     | Testing Process                                                           | Automation Level                                   | Key Challenges/Observations                                    |
|----------|--------|----------------------|---------------------------------------------------------|---------------------------------------------------------------------------|----------------------------------------------------|----------------------------------------------------------------|
| Company1 | Large  | Non-software         | Semi-automated with OAS, code generation, manual tweaks | Manual test cases, system integration level, no standard approach         | Hybrid manual and automated execution              | Lack of standardized API testing, some APIs untested           |
| Company2 | Medium | Software-oriented    | OAS, detailed definition, coding follows OAS            | Selective testing on modifications, 10% test coverage, aiming for 80%     | Manual triggers for CD, considering Jenkins        | Reactive testing, low initial coverage                         |
| Company3 | Medium | Supportive software  | OAS, pull requests, CI checks                           | Developer discretion, Post-man test suites                                | High automation in development, low in testing     | Lack of formal testing strategy, potential for more automation |
| Company4 | Small  | Software consultancy | Iterative, continuous updates, mock data                | Comprehensive testing (unit, integration, etc.), mix of tools and methods | High automation, Jenkins, Newman, manual oversight | High automation but requires manual oversight for quality      |
| Company5 | Large  | Core software        | Simulation environment, proprietary framework           | Manual during development, automated via Jenkins, 95% automated           | High test coverage (90%), aiming for 100%          | High pre-work effort, manual troubleshooting                   |

Table B.1: Summary of API Testing Practices Across Different Companies

# C

## Agent Prompts

### C.1 Team Supervisor

You are a supervisor tasked with managing a conversation between the following workers: API Spec Expert, API Spec Improver. Your team has been assigned to enhance an API specification based on user requests. Your role is to determine which worker to engage at each step of the process. The API Spec Expert is responsible for identifying areas of improvement within the API spec in natural language, focusing on clarity, documentation, and compliance without proposing functional changes. The API Spec Improver will apply these suggested improvements directly to the API specification. Begin the conversation by engaging the API Spec Expert to review the specification and list improvements. Once the API Expert has provided their suggestions, pass the task to the API Spec Improver to implement these changes into the API spec. Continue this process until all suggested improvements have been applied.

### C.2 API Expert

**Do Not Propose Functional Changes:** Avoid suggesting alterations that modify the core functionality or purpose of any endpoints. Our goal is to enhance, not to redesign.

**Avoid Altering Existing Schemas Directly:** Do not recommend removing or fundamentally changing the properties within existing request and response schemas. Suggestions should build upon, not detract from, existing structures.

**Do Not Compromise Security:** Refrain from suggesting modifications to the security schemes that could

potentially weaken or alter the existing authentication and authorization processes.

**Resist Adding New Features:** This task is not about expanding the API's capabilities with new endpoints or functions. Focus on the existing specification.

**Do Not Neglect Standards Compliance:** Any suggestions made should not lead to reduced compliance with the OpenAPI Specification (OAS) standards.

**Avoid Overcomplication:** Do not propose changes that significantly increase the complexity of the specification without a clear benefit in clarity or usability.

### C.3 API Improver

You are a crucial member of an API Enhancement team, tasked with a detailed and specific role in improving the given API specification. Your job is to apply the improvements suggested by the API Spec Expert directly to the API specification.

Please return the full answer in YAML format.

ONLY when you have covered the entire specification with the improvements, declare Task Completed: API Specification Enhanced.

### C.4 Reflection Actor

You are an API developer. You have been tasked with testing an API based on a YAML specification. Generate the best test cases possible for given API specification in JSON format. You may generate parameters for the test cases that are not part of the examples based your interpretation.

After receiving critique from the user, you should reply with the updated test cases in JSON format.

### C.5 Reflection Evaluator

You are a supervisor to an API developer, you are highly skilled in API development. Generate critique and recommendations for the developer's submission of test cases. Provide detailed recommendations, including requests for additional test cases or changes to existing ones. You may come up and recommend additional parameters.

## C.6 Test Execution Chain

Perform the following tests: {test\_cases\_to\_run}. After all tests are performed, summarize all test cases and provide feedback whether they were successful or not.

# D

## Agent Responses

### D.1 API Expert

Task Completed: Enhancement Suggestions

1. **\*\*Improve Endpoint Descriptions:\*\***
  - For ``/shopping/hotel-offers``, enhance the description to provide more context about what the endpoint does, such as "Retrieves multiple hotel offers based on the specified criteria including location, dates, and number of guests."
  - For ``/shopping/hotel-offers/{offerId}``, clarify the description to "Fetches detailed pricing and availability information for a specific hotel offer identified by the offerId."
2. **\*\*Parameter Descriptions and Examples:\*\***
  - Ensure all parameters across endpoints have clear, concise descriptions. For instance, the ``hotelIds`` parameter description could be expanded to explain how to obtain these IDs.
  - Add more ``x-example`` values for parameters where applicable, to provide users with concrete examples of valid inputs.
3. **\*\*Response Descriptions:\*\***
  - Expand on the ``200``, ``400``, and ``500`` response descriptions to include more details about what each response contains and potential reasons for errors. This could include examples of successful responses and common error responses.
4. **\*\*Schema Definitions:\*\***
  - Add more detailed descriptions to the properties within the schema definitions. For example, in the ``HotelOffers``

- definition, describe what ``available`` signifies and how ``offers`` are structured.
  - Include examples within schema definitions where missing, to illustrate what a typical object might look like.
5. **\*\*Error Handling:\*\***
    - Provide a more comprehensive guide on error handling within the API documentation section, explaining the structure of error responses and how to interpret common error codes.
  6. **\*\*Versioning and Deprecation Notices:\*\***
    - Clearly document the API versioning strategy and how clients will be informed of deprecated features or endpoints. This could be a section in the introduction or a dedicated versioning guide linked in the documentation.
  7. **\*\*Security Schemes:\*\***
    - Without altering the existing security schemes, ensure there's a section that clearly explains the authentication process, including obtaining and using access tokens.
  8. **\*\*Localization and Internationalization:\*\***
    - Offer guidance on how to use the ``lang`` parameter effectively, including a list of supported languages and default behavior if a language is not available.
  9. **\*\*Pagination and Rate Limiting:\*\***
    - If applicable, document pagination for responses that could return a large number of items. Include information on default page size, how to request specific pages, and rate limiting policies.
  10. **\*\*Best Practices:\*\***
    - Include a section on best practices for using the API, such as how to efficiently search for hotels without overloading the system and tips for parsing responses.
  11. **\*\*Interactive API Explorer:\*\***
    - If not already present, suggest integrating an interactive API explorer that allows users to try out endpoints directly from the documentation. This could greatly enhance the developer experience by providing immediate feedback.

**12. \*\*Change Log:\*\***

- Maintain a change log as part of the documentation to track and communicate changes to the API over time. This helps developers understand what has changed in each version.

**13. \*\*FAQ Section:\*\***

- Add a Frequently Asked Questions (FAQ) section addressing common queries and issues encountered by API consumers. This can reduce support requests and improve user satisfaction.

By implementing these suggestions, the API specification will become more user-friendly, informative, and easier to adopt for developers.

## D.2 Evaluator

### Review and Recommendations:

**1. \*\*Test Case Coverage:\*\***

- The test cases provided cover basic scenarios such as retrieving all categories, filtering by country, using limit and offset, handling invalid category IDs, and invalid country codes. These are good starting points for testing the API endpoints.

**2. \*\*Additional Test Cases:\*\***

- It would be beneficial to include test cases for scenarios like:
  - Testing with valid and invalid access tokens to ensure proper authentication handling.
  - Testing with different locales to verify language-specific responses.
  - Testing edge cases for limit and offset parameters (e.g., minimum and maximum values).
  - Testing for response schema validation to ensure the response structure matches the expected schema.

**3. \*\*Parameter Combinations:\*\***

- Consider adding test cases that combine multiple parameters to test the API's ability to handle various combinations effectively. For example, testing with both country and locale parameters together.

4. **\*\*Negative Testing:\*\***
  - Include negative test cases to validate error responses for scenarios like unauthorized access, rate limiting, and malformed requests.
5. **\*\*Data Validation:\*\***
  - Validate the data returned in the response against the expected data format and values, especially for fields like category IDs, country codes, and language codes.
6. **\*\*Error Handling:\*\***
  - Test the API's error handling by intentionally triggering errors, such as providing incorrect parameter values or missing required parameters.
7. **\*\*Performance Testing:\*\***
  - Consider including performance test cases to evaluate the API's response time under different load conditions, especially when using large limit values.
8. **\*\*Documentation Verification:\*\***
  - Verify that the API responses match the documented specifications to ensure consistency and accuracy.
9. **\*\*Security Testing:\*\***
  - If applicable, include security test cases to validate that sensitive data is handled securely and that the API follows best practices for data protection.
10. **\*\*Review Response Codes:\*\***
  - Double-check the expected response codes against the API documentation to ensure they align with the defined behavior.

By incorporating these recommendations and additional test cases, you can enhance the coverage and robustness of your API testing efforts. If you have any specific scenarios or need further assistance, feel free to ask for more tailored recommendations.

## D.3 Actor

### D.3.1 Negative Testing sample

---

1 {

```

2  "name": "List runs - Invalid thread ID",
3  "description": "Verify that an error is returned when
    an invalid thread ID is provided.",
4  "request": {
5      "method": "GET",
6      "url": "https://api.openai.com/v1/threads/
    invalid_thread_id/runs",
7      "headers": {
8          "Authorization": "Bearer $OPENAI_API_KEY",
9          "Content-Type": "application/json",
10         "OpenAI-Beta": "assistants=v1"
11     }
12 },
13 "expected_response_code": 404
14 }

```

### D.3.2 Boundary Testing sample

```

1  {
2      "name": "Test case with maximum limit value",
3      "request": {
4          "method": "GET",
5          "url": "https://api.spotify.com/v1/artists/{artist_id}
    /albums",
6          "headers": {
7              "Authorization": "Bearer your_access_token"
8          },
9          "params": {
10             "artist_id": "1vCWHaC5f2uS3yhpwWbIA6",
11             "limit": 50
12         }
13     },
14     "expected_response": {
15         "status_code": 200
16     }
17 }

```

### D.3.3 Error Handling sample

```

1  {
2      "name": "Get album tracks with forbidden access",
3      "request": {
4          "method": "GET",
5          "url": "https://api.spotify.com/v1/albums/4
    aawyAB9vmqN3uQ7FjRGTy/tracks",

```

```
6         "headers": {
7             "Authorization": "Bearer <
              invalid_access_token>"
8         }
9     },
10     "responseStatusCode": 403
11 }
```

### D.3.4 Concurrency Testing sample

```
1 {
2     "name": "Concurrent requests for audiobook IDs",
3     "request": {
4         "url": "https://api.spotify.com/v1/me/audiobooks/
              contains?ids=18yVqkdbdRvS24c0Ilj2ci",
5         "method": "GET",
6         "headers": {
7             "Authorization": "Bearer <valid_access_token>"
8         }
9     },
10     "expected_response_code": 200
11 }
```

### D.3.5 Performance Testing sample

```
1 {
2     "name": "Performance test for API response time",
3     "request": {
4         "method": "GET",
5         "url": "https://api.spotify.com/v1/artists/{artist_id
              }",
6         "headers": {
7             "Authorization": "Bearer your_access_token"
8         },
9         "params": {
10             "artist_id": "00dUWJ0sBjDrqHygGUXeCF"
11         }
12     },
13     "expected_response": {
14         "response_time": "<
              expected_response_time_in_milliseconds>"
15     }
16 }
```

### D.3.6 Edge Case Testing sample

```

1 {
2   "name": "List message files - Parameter Testing",
3   "description": "List message files with different
4     parameter combinations",
5   "request": {
6     "url": "https://api.openai.com/v1/threads/
7       thread_abc123/messages/msg_abc123/files?limit
8       =5&order=desc&after=obj_xyz",
9     "method": "GET"
10  },
11  "expected_response": {
12    "status": 200,
13    "schema": "#/components/schemas/
14      ListMessageFilesResponse"
15  }
16 }

```

### D.3.7 Pagination Testing sample

```

1 {
2   "name": "Get audiobook chapters for a valid ID and
3     market with pagination",
4   "request": {
5     "method": "GET",
6     "url": "https://api.spotify.com/v1/audiobooks/7
7       iHfbu1YPACw6oZPAFJtqe/chapters?market=US&limit=10&
8       offset=0",
9     "headers": {
10       "Authorization": "Bearer your_access_token"
11     }
12  },
13  "expected_response": {
14    "status_code": 200,
15    "body": {
16      "total_chapters": 20,
17      "chapters": [
18        {
19          "id": "chapter_1",
20          "title": "Chapter 1"
21        },
22        {
23          "id": "chapter_2",
24          "title": "Chapter 2"
25        }
26      ]
27    }
28  }
29 }

```

```
24     }  
25   }  
26 }
```

### D.3.8 Localization Testing sample

```
1 {  
2   "testCase": "Valid offerId with minimum length lang",  
3   "endpoint": "/shopping/hotel-offers/{offerID}",  
4   "input": {  
5     "offerId": "6PAEC60KK7",  
6     "lang": "FR"  
7   },  
8   "expectedOutput": {  
9     "statusCode": 200,  
10    "responseType": "application/vnd.amadeus+json"  
11  }  
12 }
```